# Índice


**Temas que se evalúan en el Segundo parcial** **4**
**U2: Gestión Lean-Agile de Productos de Software** **5**
1.1. Filosofía Lean 5
1.1.1. Fundamentos de Lean 5
Lean aplicado al desarrollo de software 5
Relación entre Lean y Agile 6
1.1.2. Desperdicios Lean 6
Puro desperdicio y desperdicio necesario 6
Los 7+1 desperdicios 6
1.1.3. Los 7 principios Lean 7
1.2. Método Kanban 10
1.2.1. Fundamentos de Kanban 10
Origen y relación con Lean 10
Objetivo de Kanban 11
Valores de Kanban 11
Principios de Kanban 11
1.2.2. Prácticas generales de Kanban 12
a. Visualizar el flujo de trabajo 12
b. Limitar el trabajo en proceso (WIP) 12
c. Gestionar el flujo 13
d. Hacer explícitas las políticas 13
e. Implementar ciclos de retroalimentación (feedback) 14
Mejorar colaborativamente y evolucionar experimentalmente 14
1.2.3. Funcionamiento de Kanban 15
Tablero Kanban y elementos de trabajo 15
Sistema Pull y límites WIP 15
Cuellos de botella 16
Clases de servicio y políticas 16
Políticas de calidad en los ejercicios 17
Cadencias de Kanban 17
Diferencias entre Scrum y Kanban 17
1.3. Métricas en los tres enfoques 18
1.3.1. Introducción a las métricas 18
Métricas de proceso, proyecto y producto 18
Métricas del producto 18
Relación con los enfoques 19
1.3.2. Métricas tradicionales 19
1.3.3. Métricas Agile 19

1.3.4. Métricas Lean / Kanban 20
Métricas del flujo 20
Métricas orientadas al servicio 21
Comparación entre enfoques 21
**U3: Gestión del Software como Producto 22**
2.1. Introducción a las prácticas continuas 22
2.1.1. Fundamentos 22
Objetivos y beneficios 22
Relación con la gestión de configuración y la automatización 23
2.2. Integración Continua (CI) 23
2.2.1. Concepto y funcionamiento 23
Funcionamiento general 23
Beneficios de CI 24
2.3. Entrega y Despliegue Continuos 24
2.3.1. Continuous Delivery 24
¿Cómo funciona? 24
2.3.2. Continuous Deployment 24
¿Cómo funciona? 25
Diferencia entre Continuous Delivery y Continuous Deployment 25
2.3.3. Estrategias de despliegue 25
Canary Deployment 25
Blue/Green Deployment 26
Diferencias entre ambas estrategias 26
**U4: Aseguramiento de Calidad de Proceso y Producto 27**
3.1. Fundamentos de Calidad de Software 27
3.1.1. Conceptos generales de calidad 27
Principios de calidad 27
Visiones de calidad 28
Perspectivas de calidad: necesaria, programada y realizada 28
Calidad de producto vs Calidad de proceso 29
3.1.2. Importancia de trabajar con calidad 30
3.1.3. Gestión y aseguramiento de calidad 30
Aseguramiento de Calidad de Software (SQA) 31
Control de Calidad (QC) 31
Planificación de Calidad 32
3.1.4. Actividades de aseguramiento de calidad 32
3.2. Modelos y estándares de calidad 33
3.2.1. Introducción a los modelos de calidad 33
Modelo de Calidad: de Producto vs de Proceso 33
Modelo vs Norma vs Estándar 33

3.2.2. Modelos de calidad y mejora de procesos 34
CMMI 34
Representaciones de CMMI 34
Áreas de proceso por nivel y categoría 35
SPICE 36
IDEAL 36
Relación entre CMMI, SPICE e IDEAL 36
3.2.3. Calidad de procesos en diferentes enfoques 37
Enfoque tradicional de mejora de procesos 37
Relación entre CMMI y Agile 38
Semejanzas y diferencias entre enfoques 38
Relación con Lean/Kanban 39
3.3. Auditorías y revisiones técnicas 39
3.3.1. Auditorías de software 39
Objetivos y beneficios 39
Tipos de auditorías 40
Auditoría al Grupo de Aseguramiento de Calidad (GAC) 40
Roles y responsabilidades 40
3.3.2. Proceso de auditoría 41
a. Preparación y planificación 41
b. Ejecución 41
c. Análisis y reporte de resultados 41
d. Seguimiento 41
Hallazgos de una auditoría (resultados obtenidos al evaluar evidencias) 42
Técnicas y herramientas 42
Métricas de auditoría 42
3.3.3. Revisiones técnicas 44
Verificación y Validación (V&V) 44
Principios de Verificación y Validación 44
Objetivos, ventajas y desventajas 45
Tipos de revisiones técnicas 45
Walkthrough (recorrida informal) 45
Inspección de software (formal) 46
Proceso de inspección formal y responsabilidades 46
Etapas 46
Técnicas y herramientas 47
3.3.4. Comparación de actividades de calidad 48
Diferencias fundamentales 49
Ejemplo (desarrollo de un sistema bancario) 49
3.4. Testing de Software 50

3.4.1. Fundamentos de testing 50
Principios de testing (7) 50
Error, defecto y falla 50
Severidad vs Prioridad 51
¿Cuánto testing es suficiente? 51
Proceso de pruebas 52
Ciclo de Prueba 52
Ambientes de testing 53
Plan de pruebas y Casos de prueba (artefactos fundamentales) 53
Elementos esenciales del caso de prueba 53
3.4.2. Niveles y tipos de testing 53
Niveles de prueba 54
Pruebas unitarias (componentes) 54
Pruebas de integración (interfaces) 54
Pruebas de sistema 54
Pruebas de aceptación 55
Tipos de prueba 56
Modelo en V (y relación con niveles de prueba) 57
3.4.3. Estrategias y métodos de testing 58
Caja blanca 58
Caja negra 58
Diferencias entre caja blanca y caja negra 60
3.4.4. Testing en ambientes Agile 60
Testing Tradicional vs. testing Agile 61
Manifiesto de Testing Agile 61
Principios de Testing Agile 62
Prácticas de Testing Agile 62
Pruebas exploratorias y de regresión 62
Rol y responsabilidades del tester ágil 63
Cuadrantes de Testing Ágil 63
Pirámide del Testing Agile 64
Automatización del testing 65
Desafíos culturales del Testing Agile 65
**4. Parte práctica: Testing de Caja Negra 66**
4.1. Partición de equivalencias 66
4.1.1. Fundamentos 66
4.1.2. Clases de equivalencia de entrada 66
4.1.3. Clases de equivalencia de salida 66
4.2. Diseño de casos de prueba 66
4.2.1. Derivación de casos de prueba 66

4.2.2. Estructura de un caso de prueba 66
4.3. Procedimiento para resolver el parcial práctico 66
4.3.1. Resolución paso a paso 66

# Temas que se evalúan en el Segundo parcial

## Unidad 2: Gestión Lean Agile de Productos de Software

- Filosofía Lean - Método Kanban en el contexto del desarrollo de software.
- Métricas en los 3 enfoques (Tradicional / Lean / Agile).

## Unidad 3: Gestión de software como producto

- Prácticas continuas.

## Unidad 4: Aseguramiento de Calidad de Proceso y de Producto

- Conceptos generales sobre calidad.
- Importancia de trabajar para y con Calidad. Ventajas y Desventajas.
- Actividades relacionadas con el Aseguramiento de la Calidad del Software.
- Principales Modelos de Calidad existentes (CMMI – SPICE – ISO).
- Auditorías al Software: Auditorías de Proyecto y Auditorías al Grupo de Calidad.
- Proceso de Auditorías
- Calidad de Producto: Planificación de pruebas para el software- Niveles y tipos de pruebas para el software. Técnicas y herramientas para probar software.
- Técnicas y Herramientas para la realización de revisiones técnicas del software.
- Testing en ambientes Ágiles.

# U2: Gestión Lean-Agile de Productos de Software

---

## 1.1. Filosofía Lean

### 1.1.1. Fundamentos de Lean

**Lean** es una filosofía de gestión originada en el Sistema de Producción de Toyota, cuyo objetivo es **maximizar el valor entregado al cliente y minimizar los desperdicios**, mediante la mejora continua de los procesos.

Se fundamenta en dos conceptos principales:

- **Valor:** viene desde el cliente y es aquello que satisface una necesidad real del mismo.
- **Desperdicio:** toda actividad que consume recursos (tiempo, esfuerzo o dinero) sin aportar valor al cliente.

El principio central de Lean es **eliminar desperdicios**, buscando que el trabajo fluya eficientemente y optimizando el proceso completo, no solamente sus actividades individuales.

#### Lean aplicado al desarrollo de software

La pareja Poppendieck trajo la filosofía Lean (Lean development) desde la manufactura por parte de empresas automovilísticas, especialmente Honda y Toyota, al mundo del Software.

En software, Lean busca concentrar el conocimiento **en entregar soluciones que satisfagan necesidades reales del cliente, evitando actividades innecesarias**, desde que surge dicha necesidad hasta que el usuario recibe una solución. Esto implica:

- Evitar desarrollar funcionalidades que el cliente no necesita.
- Reducir esperas, retrabajos y trabajo sin terminar.
- Entregar software útil lo antes posible.
- Mejorar continuamente el proceso de desarrollo completo.

**Ejemplo:** desarrollar funcionalidades adicionales que nadie solicitó ni necesita consume tiempo y esfuerzo sin generar valor, por lo que representa un desperdicio. *Encima si le decimos al cliente acerca de las nuevas funcionalidades que **podemos** agregarle, ya los convencemos de algo que no le aporta valor pero que, si ahora no lo tiene, le pueda generar insatisfacción.*

### Relación entre Lean y Agile

Lean y Agile son enfoques compatibles que comparten la **entrega de valor al cliente, el aprendizaje continuo y la adaptación mediante retroalimentación (mejora continua)**.

Por un lado, **Lean** propone principios universales que orientan cómo maximizar el valor y eliminar desperdicios. Por el otro tenemos **Agile**, que también posee valores y principios propios, pero ofrece métodos y prácticas concretas para desarrollar software de manera adaptable al contexto.

Así, los principios Lean pueden materializarse mediante enfoques ágiles, por ejemplo:

- **Amplificar el aprendizaje:** mediante retrospectivas (transparencia) y feedback frecuente.
- **Dar poder al equipo:** mediante equipos autogestionados, responsables y reactivos.
- **Entregar lo antes posible:** mediante entregas pequeñas y frecuentes.

## 1.1.2. Desperdicios Lean

Un **desperdicio** es aquella actividad que consume recursos (costo, tiempo, esfuerzo) sin aportar valor al cliente, ni directa ni indirectamente.

### Puro desperdicio y desperdicio necesario

Lean clasifica en dos tipos los desperdicios, dependiendo de la actividad. No propone eliminar todas las actividades que no generen valor directamente, sino distinguir cuáles son **necesarias** y cuáles representan **puro desperdicio**.

- **Puro desperdicio:** actividad que no aporta valor al cliente, **ni es necesaria** para el proceso. Es la razón por la que se busca **eliminar** completamente.

- **Desperdicio necesario:** actividad que no aporta valor de forma directa al cliente pero que *no puede eliminarse dadas las condiciones actuales*, es decir, que es necesaria para el proceso. Es la razón por la que se busca **minimizar** y no eliminar.

### Los 7+1 desperdicios

Clasificación de 8 tipos de actividades que consumen recursos sin aportar valor real al cliente. Originalmente Lean identificaba 7 desperdicios en manufactura. Posteriormente se incorporó un octavo: **talento no utilizado**. 7+1 para conservar la clasificación original, destacando el octavo.

Los desperdicios Lean permiten identificar dónde se pierde tiempo, esfuerzo o recursos dentro del proceso. Su objetivo es **eliminar el puro desperdicio y minimizar el desperdicio necesario**, mejorando el flujo de trabajo y la entrega de valor.

| Manufactura | Desarrollo | Ejemplos |
|---|---|---|
| Inventario | Trabajo no terminado (gris) | O blanco o negro. No aporta valor al cliente un requisito que no se terminó de desarrollar. |
| Sobreproducción | Features extra | Aquello que no nos pidió el cliente pero “creemos que puede aportar valor”. |
| Proceso extra | Proceso extra | Pasos innecesarios generados en un proceso, como puede ser documentación excesiva o aprobación innecesaria que frena el flujo. |
| Transporte | Transferencia de conocimiento (Handoff) | Poca transparencia en un equipo, que genera retrabajo; Capacitar gente nueva en el equipo. |
| Esperas | Demoras | Demora de revisiones técnicas, del PO, pasos burocráticos, necesidad de validación, etc. |
| Movimiento | Context Switching | Agregar a una persona a más de un proyecto, interrumpiendo y haciendo que pierda el foco. |
| Defectos | Defectos | Generar algo incorrecto que nos de retrabajo. |
| **Talento no utilizado** | **Talento no utilizado** | Tener una persona sobrecalificada en un puesto, generando que se aburra y se vaya; Que un buen dev se vaya como Team leader por el sueldo. |

### 1.1.3. Los 7 principios Lean

Los **principios** son más universales y se pueden extrapolar a diferentes dominios/contextos, en cambio las **prácticas** son una especie de guía de cómo se debería llevar a cabo algo, lo que están bastante más ligadas al dominio, por lo que se proponen principios.

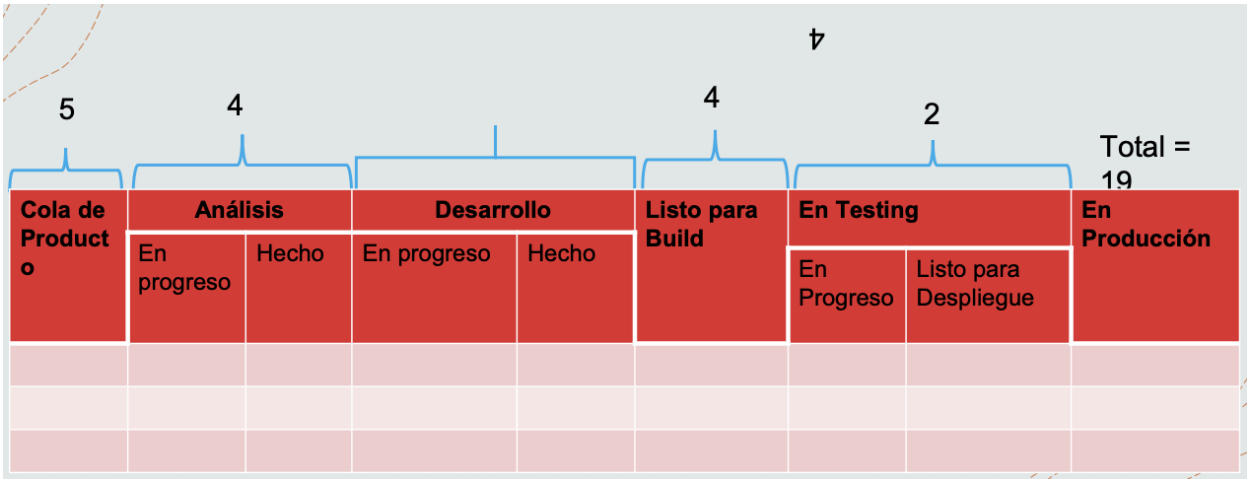
- **Eliminar desperdicios.** Acumular requerimientos o funcionalidades que no serán usadas es un desperdicio, al igual que desarrollar funcionalidades que no serán utilizadas, el traspaso de conocimiento (ejemplo cuando se recluta gente nueva, o cuando se va un desarrollador que sabía todo y no dejó documentación), o todo aquello que no permite entregar software rápidamente al cliente (relacionado a principio entregar lo antes posible).
    - Relación con todos los demás principios.
- **Amplificar el aprendizaje.** El desarrollo de software es una **actividad basada en conocimiento**, donde muchas veces tenemos **incertidumbre** y no conocemos cuales son las necesidades del cliente ni cuál es la mejor solución que podemos plantear desde el principio. Es por eso que Lean propone aprender continuamente para reducir dicha incertidumbre y tomar mejores decisiones. Algunas cosas que se pueden hacer:

equipos autogestionados, revisiones entre pares, iteraciones cortas (ir disminuyendo el cono de la incertidumbre) para ir adaptándonos, feedback frecuente para detectar errores rápidamente.
    - Relación con eliminar desperdicios, diferir compromisos, entregar lo antes posible y dar poder al equipo.
- **Diferir compromisos (decidir lo más tarde posible).** Es uno de los principios que dice que **posponer** las decisiones críticas para el último momento responsable (cuando podemos decidir sin perjudicar el desarrollo ni perder alternativas) sin que se vea afectado el proceso de desarrollo es óptimo, dado que permite no tomar malas decisiones en aquellas actividades que son críticas y que luego podrían generar defectos en etapas posteriores (retrabajo, costos, retrasos, entre otros). Relacionado al cono de la incertidumbre con el que se comienza en el desarrollo de software, cuanto más iteramos más vamos a ir aprendiendo aquellos requerimientos emergentes del software.
    - Relación con eliminar desperdicios, amplificar el aprendizaje, ver el todo.
- **Entregar lo antes posible.** Básicamente antes se pensaba que entregar una vez sin ningún error era lo más valioso, pero esto cambió debido a que entra un concepto de que la velocidad cuesta más, es decir que… en el desarrollo de software el ciclo de descubrimiento es lo **crítico para el aprendizaje**, dado que necesitamos diseñar, implementar obtener feedback para poder mejorar y adaptar el sistema a las necesidades reales, no a lo que suponemos. Cuanto más rápido sean estos ciclos de feedback, más aprenderemos de cual es la necesidad del cliente y el camino por el que tiene que ir nuestro sistema. La velocidad asegura que el cliente pueda recibir lo que realmente necesita y espera hoy, no lo que necesitaba en el momento que tomamos los requerimientos (que puede ser hace mucho tiempo si es un ciclo no iterativo o con iteraciones largas/poco frecuentes), esto debido a que se puede **validar antes las necesidades del mismo y adaptarse rápidamente**.
    - Relación con eliminar desperdicios (clave), amplificar el conocimiento, diferir compromisos (mantener abierta decisiones importantes y tomarlas con mayor conocimiento iterando mucho hasta obtenerlo), embeber la integridad conceptual (no se debe comprometer la calidad del software por hacerlo rápido).
- **Dar poder al equipo.** la inteligencia colectiva del equipo permite tomar mejores decisiones a que si uno solo fuera el responsable de ello. Cuando un equipo está guiado por un líder que entiende esto permite que se tomen mejores decisiones técnicas en conjunto. Equipos **autogestionados** comprometidos con su trabajo y con el fin propuesto. Mejorar la **señalización o diferentes caminos de comunicación** permiten que el trabajo entre los miembros de un equipo esté coordinado, sin depender constantemente de un **jefe** (jefe vs líder). En entornos donde las cosas cambiar rápidamente, no es eficiente (genera desperdicio) que cada decisión tenga que pasar por un responsable jerárquico -> dar autonomía + mecanismos de señalización.
    - Relacionado con eliminar desperdicio, amplificar aprendizaje, entrega lo antes posible (fomenta la rápida resolución, ejemplo con colaboración entre tasks de

un tablero), ver el todo (señalización que perita comunicar y coordinar, evitando que haya optimizaciones individuales sino que sean en equipo).

- **Embeber la integridad conceptual.** Un buen proceso de calidad no garantiza tener automáticamente un producto de calidad. Los factores de los que la calidad es más dependiente son: **liderazgo adecuado, compromiso del equipo, expertise relevante y comunicación efectiva**. Estos factores no pueden ser reemplazados por construir un buen proceso de calidad, tener métricas excelentes y procesos documentados… ¿Qué pasa si hay una comunicación deficiente en el equipo? Los procesos maduros (CMMI, SPICE e IDEAL) no hacen que un producto satisfaga las necesidades reales del cliente -> necesidad de procesos que maximicen el valor y reduzcan desperdicios como actividad innecesaria. Básicamente Lean no los rechaza sino visibiliza que cumplir con estos no produce automáticamente calidad.
    - Para que tenga integridad debe satisfacer las **necesidades del usuario**. Un producto de software no puede tener calidad si no tiene integridad ni viceversa: La calidad no se negocia.
    - Se relaciona con eliminar desperdicios, entregar lo más rápido posible (obtener feedback que haga a la calidad del cliente pero sin sacrificarla), amplificar aprendizaje (revisiones walkthrough informales pero existentes), tomar decisiones tardías (permite que tomarlas en un tiempo razonable que no sea perjudicial pueda amplificar el conocimiento y que tenga aún más calidad tanto el proceso como el producto final), ver el todo (la integridad requiere que todas las partes del software sean coherentes entre sí y funcionen correctamente como un sistema completo).
- **Ver el todo.** Optimizar el sistema completo en vez de enfocarse en la optimización de sus partes en sistemas aislados. Esto hace que una parte del equipo, ejemplo: el que se encarga de la base de datos, sea muy productivo pero se genere un cuello de botella porque el de GUI no puede terminar el trabajo (está parcialmente completado). Esto hace que se generen esperas y trabajo no terminado, derivando en retrasos o reasignación de user stories a otro Sprint (o que el PO las despriorice y las vuelva al PB, haciendo que haya sido un desperdicio por no terminar el trabajo) si estamos trabajando en Scrum.
    - Se relaciona
    - Luego podrá ser implementado el tema de poder trabajar conjuntamente con el equipo en caso de tener tareas en columna de listo en una etapa (KANBAN) para poder pasarlas a WIP. Es decir, colaboración entre las diferentes partes del equipo. EL WIP TIENE QUE DEFINIRSE EXPLÍCITAMENTE MEDIANTE UNA

POLÍTICA DE CUANTOS WIP PUEDEN HABER EN CADA COLUMNA.




- Se puede relacionar a Agile cuando se trabaja con user verticales también creo. Dado que hace que un requisito del sistema se tenga que desarrollar completo y no quede parcialmente terminado (en caso de si terminar la user). **VER**

# 1.2. Método Kanban

## 1.2.1. Fundamentos de Kanban

**Kanban** es un método para definir, gestionar y mejorar servicios **basados en trabajo del conocimiento**, como el desarrollo de software, donde intervienen la creatividad, el diseño y la resolución de problemas.

Su idea central es **gestionar y optimizar el flujo continuo de trabajo**, haciendo visible cómo las tareas avanzan por las distintas etapas del proceso.

> **A diferencia de Scrum**, Kanban no exige organizar el trabajo en Sprints de duración fija, sino que permite gestionar un flujo continuo de actividades.

### Origen y relación con Lean

Kanban tiene sus antecedentes en el **Sistema de Producción de Toyota**, donde se utilizaban tarjetas de señalización para controlar la producción y el inventario bajo el enfoque *Just in Time* (producir lo necesario, cuando se necesita). David J. Anderson desarrolló el **Método Kanban**.

Kanban se relaciona con Lean porque aplica sus ideas de **eliminar desperdicios, mejorar continuamente y optimizar el flujo completo de trabajo**, buscando entregar valor al cliente.

### Objetivo de Kanban

El objetivo de Kanban es **lograr un flujo continuo de trabajo fluido, predecible y sostenible**, entregando valor al cliente de manera continua. Para ello busca:

- **Visualizar el flujo de trabajo:** conocer el estado de las actividades y detectar problemas.
- **Limitar el trabajo en proceso (WIP):** evitar la sobrecarga y acumulación de tareas.
- **Gestionar el flujo:** identificar cuellos de botella y reducir demoras.
- **Mejorar continuamente:** adaptar el proceso a partir de la observación y el feedback.

### Valores de Kanban

Este método se fundamenta en nueve valores:

- **Transparencia:** compartir información abiertamente.
- **Equilibrio:** balancear la demanda con la capacidad disponible.
- **Colaboración:** trabajar conjuntamente para mejorar.
- **Foco en el cliente:** orientar el trabajo a sus necesidades.
- **Flujo:** facilitar el movimiento continuo del trabajo. No hay proyecto dentro de Kanban, por lo que las tareas deben llevarse a cabo mediante un flujo continuo. Scrum maneja time boxes (hay un inicio y un fin), en Kanban **el flujo siempre es continuo**.
- **Liderazgo:** promover iniciativas de mejora en todos los niveles.
- **Entendimiento:** conocer cómo funciona el sistema actual.
- **Acuerdo:** comprometerse conjuntamente con la mejora.
- **Respeto:** valorar a las personas y sus contribuciones. El respeto es el valor fundamental que **sostiene a los demás**.

### Principios de Kanban

Se organizan en dos grupos principalmente:

1. **Principios de gestión del cambio**
    - **Comenzar con lo que haces ahora:** comprender y respetar los procesos, roles y responsabilidades actuales, sin imponer cambios radicales. Luego se refinará.
    - **Acordar la búsqueda de mejora mediante cambios evolutivos:** introducir mejoras pequeñas y progresivas.
    - **Fomentar actos de liderazgo a todos los niveles:** cualquier integrante puede impulsar mejoras.

2. **Principios de entrega de servicios**
    - **Comprender y enfocarse en las necesidades y expectativas del cliente.**
    - **Gestionar el trabajo y permitir que las personas se organicen alrededor de él.**

- **Revisar periódicamente los servicios y sus políticas para mejorar los resultados.**

### 1.2.2. Prácticas generales de Kanban

Las **6 prácticas generales de Kanban** permiten gestionar y mejorar continuamente el flujo de trabajo, buscando que sea fluido, predecible y sostenible.




#### a. Visualizar el flujo de trabajo

Consiste en **representar visualmente el trabajo y las etapas por las que atraviesa**, lo cual permite conocer su estado, identificar bloqueos y detectar oportunidades de mejora. En general, se utiliza un **tablero Kanban**, cuyas columnas representan etapas del proceso y cuyas tarjetas representan elementos de trabajo.

- Facilita la transparencia, coordinación y comunicación con el equipo.
- Permite identificar tareas bloqueadas, acumulaciones y cuellos de botella.
- Ayuda a tomar decisiones rápidas basadas en el estado real del trabajo.

| |
|---|
| Permite identificar las actividades del proceso y representarlas en columnas, adaptándolas al contexto del equipo. |

#### b. Limitar el trabajo en proceso (WIP)

Consiste en **establecer límites explícitos sobre la cantidad de elementos de trabajo** que pueden estar en una etapa del proceso (columna) simultáneamente. Su propósito es evitar la sobrecarga, la multitarea excesiva y la acumulación de trabajo “parcialmente terminado”.

- **Conversación y mejora:** permite fomentar la comunicación y la colaboración.
- **Deja de empezar y empezar a terminar:** no iniciar tareas sin finalizar las anteriores.
- **Limitar WIP según capacidad real del sistema**, ajustando mediante observación y datos.
- **Favorecer un sistema Pull:** incorporar nuevo trabajo cuando hay capacidad disponible.

| |
|---|
| Establecer límites WIP considerando la cantidad de personas, la complejidad y la capacidad |

de cada etapa. No necesariamente existe una relación de una tarea por persona.

**Ejemplo de limitar WIP:** Si el desarrollo tiene WIP de 2 y hay una tarea en progreso y otra hecha, no se puede comenzar una tercera hasta liberar capacidad (que la hecha pase a próxima etapa).

### c. Gestionar el flujo

Consiste en **observar, medir y mejorar el movimiento del trabajo** desde que ingresa al sistema hasta su entrega, buscando un flujo continuo, predecible y confiable. El objetivo no es mantener a todas las personas ocupadas al máximo, sino optimizar el rendimiento del sistema completo.

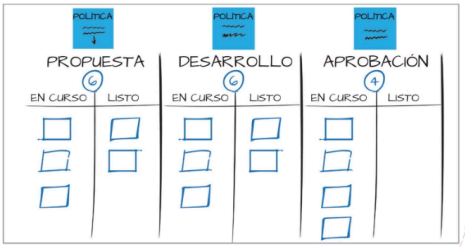
- Identificar y resolver cuellos de botella.
- Reducir esperas, bloqueos y acumulaciones.
- Utilizar datos y métricas para mejorar la previsibilidad y tomar decisiones.

Si Testing acumula muchas tareas esperando ser probadas, se debe analizar esa etapa y buscar cómo recuperar el flujo, en lugar de continuar aumentando la producción de desarrollo (una posible puede ser la colaboración de derecha a izquierda).

### d. Hacer explícitas las políticas

Consiste en **definir, acordar y hacer visibles las reglas** que determinan cómo se gestiona y avanza el trabajo, para que todas las personas comprendan y respeten el funcionamiento del sistema. Estas políticas deben ser **claras, simples, visibles y modificables** cuando se identifiquen oportunidades de mejora.

Las políticas deben estar expuestas en un lugar destacado, eso es lo que hace referencia la **explicitación** de las políticas. Estas deben ser acordadas entre todas las partes involucradas, incluyendo clientes, interesados y los que harán el trabajo.




Pueden establecer:

- **Criterios de entrada y salida:** condiciones necesarias para avanzar una tarea entre etapa
- **Límites WIP:** cantidad máxima de elementos en progreso. Explícitos en el tablero.
- **Criterios Pull:** cuándo y bajo qué condiciones incorporar nuevo trabajo (traer tarea de una etapa anterior a la actual que haya finalizado).
- **Clases de servicio:** cómo tratar elementos con diferentes necesidades o prioridades.
- **Políticas de calidad:** condiciones que deben cumplirse, como revisiones técnicas, pruebas o aprobaciones.

| |
|---|
| Definir reglas concretas según las condiciones de contexto. |

**Ejemplo de hacer política explícita:** una funcionalidad no puede pasar a Testing hasta que el código haya sido revisado y aprobado por otro desarrollador.

### e. Implementar ciclos de retroalimentación (feedback)

Consiste en **establecer instancias periódicas de revisión** para evaluar el funcionamiento del sistema, compartir información y detectar oportunidades de mejora. Estas instancias se hacen con una **cadencia adecuada**, es decir, con una frecuencia establecida según las necesidades del equipo y del servicio.

- Favorecen la colaboración, el aprendizaje y la mejora continua.
- Permiten revisar bloqueos, riesgos, entregas y resultados.
- Utilizan información real para orientar decisiones y ajustes.

| |
|---|
| Realizar reuniones breves y periódicas para revisar el flujo, identificar problemas y coordinar acciones. |

**Ejemplo de implementar ciclos de feedback:** reunión Kanban diaria con el equipo para analizar qué tareas están bloqueadas y cómo ayudar a terminarlas.

### Mejorar colaborativamente y evolucionar experimentalmente

Consiste en **mejorar continuamente el proceso** mediante cambios pequeños, colaborativos y basados en hipótesis, evaluando sus resultados antes de adoptarlos definitivamente. Kanban propone comenzar con el proceso actual y evolucionarlo progresivamente, utilizando un enfoque experimental:

1. Identificar un problema u oportunidad de mejora.
2. Proponer una hipótesis de cambio.
3. Realizar un experimento controlado y seguro para fallar (*safe-to-fail*).
4. Medir y evaluar los resultados.
5. Mantener, ajustar o revertir el cambio según lo aprendido.

| |
|---|
| Experimentar con modificaciones en las políticas, los límites WIP o la organización del trabajo para comprobar si mejoran el flujo. |

**Ejemplo de mejora y evolución experimental:** reducir temporalmente el límite de WIP en desarrollo de 4 a 3 para evaluar si disminuye la acumulación de tareas pendientes de Testing.

## 1.2.3. Funcionamiento de Kanban

### Tablero Kanban y elementos de trabajo

El **tablero Kanban** es una representación visual del proceso de trabajo, adaptada al contexto del equipo y del servicio que se presta. Está compuesto por diferentes elementos:

- **Columnas:** representan las etapas o estados del flujo de trabajo (Análisis, Desarrollo, Testing, Despliegue, etc.).
- **Tarjetas:** representan los elementos de trabajo que avanzan por el sistema (funcionalidades, defectos, tareas, etc.).
- **Subcolumnas:** permiten distinguir estados dentro de una etapa **(En progreso y Listo)**.
- **Carriles (swimlanes):** divisiones horizontales opcionales para organizar el trabajo por tipo, cliente o clase de servicio.

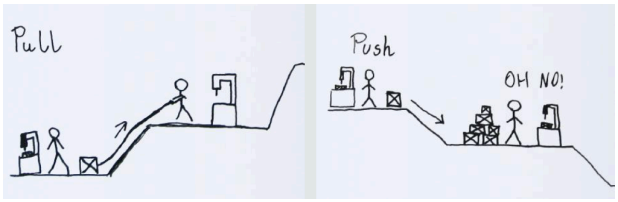
El tablero debe reflejar **cómo trabaja realmente el equipo**, incluyendo sus actividades, esperas, aprobaciones y controles de calidad.

| |
|---|
| Para diseñar un tablero, primero se identifican las actividades necesarias según el contexto, los integrantes y los acuerdos establecidos. |

**Ejemplo:** si el enunciado exige revisiones técnicas antes de Testing y aprobación del cliente antes del despliegue, ambas condiciones deben quedar representadas en el flujo (tablero) o mediante políticas explícitas.

### Sistema Pull y límites WIP

Kanban utiliza un **sistema Pull (arrastre)**: cada etapa toma trabajo de la anterior, siempre y cuando tiene capacidad disponible y se cumplen los criterios de entrada.




*Se diferencia de un sistema Push (empuje), donde una etapa envía trabajo a la siguiente sin necesariamente considerar su capacidad.*

Los **límites WIP** establecen la cantidad máxima de elementos que pueden encontrarse simultáneamente en determinadas columnas, conjuntos de columnas o en el sistema completo. Para esto se recomienda: considerar capacidad disponible, cantidad de personas asignadas, si el equipo es multidisciplinario, entre otras cosas.

### Cuellos de botella

Un **cuello de botella** es una etapa cuya capacidad limita el flujo de trabajo del sistema completo, provocando acumulaciones y demoras. Ejemplo, *Desarrollo termina 5 funcionalidades por día, pero Testing solamente puede verificar 2 –> las tareas se acumularán antes de Testing.*

Para gestionarlo, Kanban propone:

- Identificar dónde se acumula trabajo o aumentan las esperas.
- Evitar producir más trabajo del que las etapas siguientes pueden procesar.
- Colaborar para resolver el problema, cuando sea posible.
- Ajustar políticas y límites WIP según los resultados observados (mejora experimental).

| |
|---|
| Optimizar una etapa de forma aislada no necesariamente mejora el sistema completo. El objetivo es optimizar el flujo de extremo a extremo (**Ver el todo**, principio Lean). |

### Clases de servicio y políticas

Las **clases de servicio** permiten gestionar elementos de trabajo con diferentes niveles de urgencia, riesgos y consecuencias por retrasarse. Las cuatro clases habituales son:

| Clase | Característica | Tratamiento |
|---|---|---|
| **Expreso (Expedite)** | Trabajo de urgencia muy alta. | Prioridad excepcional; puede interrumpir otros trabajos según las políticas. |
| **Fecha fija** | Debe completarse antes de una fecha determinada. | Se planifica y supervisa para evitar el incumplimiento. |
| **Estándar** | Trabajo normal, sin urgencia extraordinaria. | Sigue las reglas habituales de priorización y flujo. |
| **Intangible** | Beneficio o consecuencia de demora menos evidente en el corto plazo. | Se planifica para evitar postergarlo indefinidamente. |




Cada clase de servicio puede tener **políticas explícitas diferentes**, como prioridades, criterios de selección, límites WIP o condiciones de entrega. Es necesario no confundir:

- **Tipo de trabajo:** qué se realiza (funcionalidad, defecto, mejora técnica, contenido).
- **Clase de servicio:** cómo se debe tratar ese trabajo según su urgencia, riesgo o costo de retraso.

Políticas de calidad en los ejercicios

Son reglas que establecen las condiciones necesarias para asegurar la calidad del trabajo al avanzar por el flujo. Ejemplos:

- Todo cambio de código debe superar una revisión técnica antes de ingresar a Testing.
- Una funcionalidad debe superar las pruebas definidas antes de considerarse lista para despliegue.
- El referente del cliente debe validar la funcionalidad antes de su puesta en producción.
- Un elemento no puede ingresar a una etapa si supera su límite WIP.
- Los defectos críticos deben identificarse y gestionarse según una política de prioridad explícita.

Las políticas deben ser **pocas, claras, visibles, acordadas y aplicables**, y estar alineadas con el contexto del servicio.

### Cadencias de Kanban

Las **cadencias** son reuniones o revisiones periódicas utilizadas para coordinar el trabajo, obtener feedback y mejorar la prestación del servicio. Su frecuencia se adapta al contexto y no depende de Sprints obligatorios. *Kanban identifica siete cadencias: Revisión de estrategia, de operaciones, de riesgos, de prestación del servicio, de reposición o realimentación, reunión Kanban y planificación de entrega.*

| |
|---|
| No es obligatorio realizar siete reuniones independientes. Pueden combinarse o adaptarse según las necesidades de la organización. |

### Diferencias entre Scrum y Kanban

| Scrum | Kanban |
|---|---|
| Framework ágil | Método de gestión y mejora del flujo |
| Organiza el trabajo en Sprints de duración fija (time-box) | Gestiona un flujo continuo de trabajo, que no tiene inicio ni final. Limitado por WIPs |
| Define responsabilidades, eventos y artefactos | No prescribe roles ni eventos obligatorios |
| Planifica el trabajo del Sprint | Incorpora trabajo según políticas Pull y la capacidad disponible |
| Inspecciona y adapta mediante eventos | Mejora mediante feedback, métricas y cambios evolutivos |

# 1.3. Métricas en los tres enfoques

## 1.3.1. Introducción a las métricas

Una **métrica de software** es una característica del producto, proyecto o proceso que puede medirse de manera objetiva. Permite obtener información cuantitativa para evaluar y tomar decisiones.

Se llevan a cabo estas mediciones para conocer el estado del trabajo, evaluar la calidad, comparar resultados e identificar oportunidades de mejora. Las métricas deben aportar información útil: **medir lo necesario y mantenerlo simple**.

Las métricas ayudan a balancear las necesidades y expectativas de los interesados con el alcance, el tiempo y los recursos, considerando la calid**ad**. Este balance está condicionado por las restricciones de **tecnología, personas y procesos**.

### Métricas de proceso, proyecto y producto

- **Proceso:** permiten evaluar y mejorar la forma de trabajar. Las métricas de distintos proyectos se consolidan para obtener información del proceso a nivel organizacional.
- **Proyecto:** permiten gestionar un proyecto particular y comparar lo planificado con lo real. Ejemplos: esfuerzo, tiempo calendario y costos.
- **Producto:** describen características del software desarrollado. Ejemplos: tamaño, defectos y cobertura.

La diferencia depende de **qué se quiere evaluar**: la forma de trabajar, un proyecto concreto o el software producido.

### Métricas del producto

- **Tamaño:** expresa la dimensión del producto. Puede medirse mediante líneas de código, requerimientos, casos de uso, features, clases o puntos de historia, según el contexto.

- **Densidad de defectos:** relaciona los defectos encontrados con el tamaño del producto: `cantidad de defectos / unidad de tamaño`. Debe indicarse la unidad utilizada para interpretar el resultado.

- **Cobertura:** expresa qué proporción de los elementos definidos como objetivo abarcan las pruebas. Se calcula como `elementos cubiertos / total de elementos × 100`. Siempre debe aclararse qué se está cubriendo.

### Relación con los enfoques

El enfoque **Tradicional** presenta métricas como esfuerzo, tiempo, costos y defectos. En **Agile**, se prioriza el software funcionando y se mide solo lo necesario. En **Lean/Kanban**, se mide principalmente el flujo de trabajo y su eficiencia. Las métricas de producto se presentan como una dimensión adicional a esas métricas de gestión.

## 1.3.2. Métricas tradicionales

- **Tamaño del producto → producto:** expresa la dimensión del software desarrollado. Puede medirse en líneas de código, requerimientos, casos de uso u otras unidades.
- **Esfuerzo → proyecto:** cantidad de trabajo humano dedicado a una actividad o al proyecto. Puede expresarse en horas-persona o días-persona.
- **Tiempo calendario → proyecto:** duración transcurrida entre el inicio y el fin de una tarea, etapa o proyecto. Permite comparar las fechas y duraciones estimadas con las reales.
- **Costos → proyecto:** recursos económicos necesarios o utilizados para desarrollar el proyecto. Se compara el presupuesto planificado con el gasto real.
- **Defectos → producto:** cantidad y tipo de defectos encontrados en el software. Pueden clasificarse por severidad y analizarse según dónde se detectaron: revisiones, pruebas unitarias, integración o producto entregado.

**Esfuerzo y tiempo calendario no son lo mismo:** dos personas trabajando durante tres días pueden representar **seis días-persona de esfuerzo**, aunque hayan transcurrido **tres días calendario**.

Estas métricas pueden aportar información sobre el **proceso** cuando se consolidan los resultados de distintos proyectos para evaluar y mejorar la forma de trabajar de la organización.

## 1.3.3. Métricas Agile

En **Agile**, basado en procesos empíricos, el **software funcionando es la principal medida de progreso**. Se busca medir solo lo necesario, obteniendo las métricas como resultado del trabajo.

- **Velocidad (Velocity):** cantidad de trabajo que el equipo **completó en un Sprint**. Se calcula sumando los **puntos de historia de las historias terminadas**, que cumplen la DoD (incluyendo las pruebas acordadas). Valor calculado que va servir como referencia para planificar los próximos Sprints.

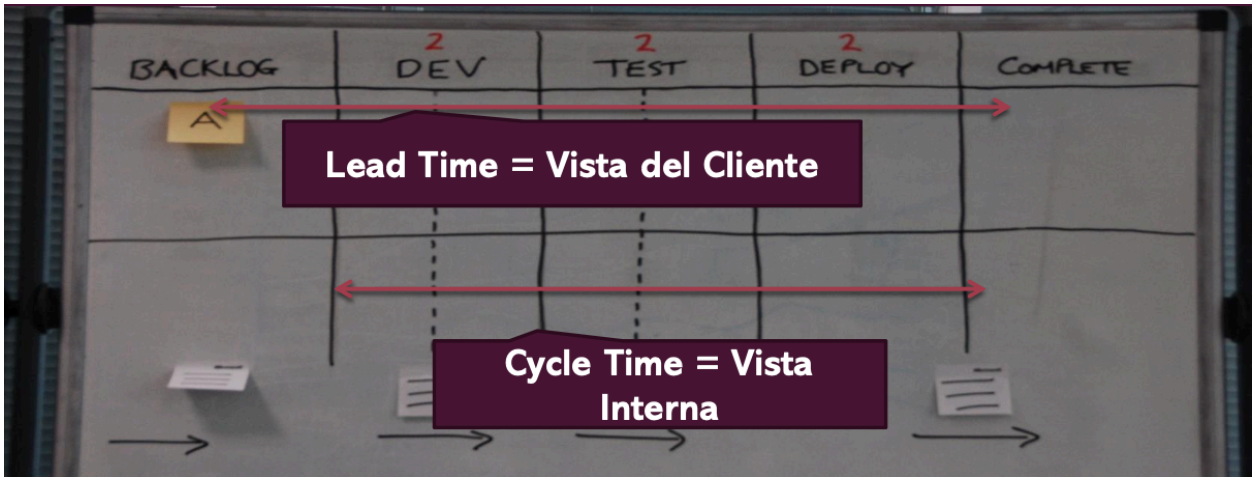
- **Capacidad (Capacity):** disponibilidad del equipo para trabajar durante **un Sprint** concreto. Se estima considerando las personas disponibles y el tiempo que pueden dedicar, descontando ausencias, feriados y otras obligaciones. Puede expresarse en **horas-persona disponibles** y ayuda a determinar cuánto trabajo puede asumir el equipo.

- **Running Tested Features (RTF):** cantidad acumulada de features implementadas que **funcionan y superan sus pruebas**. Permite visualizar el avance del producto mediante software funcionando y probado. Se relaciona con la **Definition of Done**: programar una funcionalidad no basta para considerarla terminada si todavía faltan las pruebas acordadas.

### 1.3.4. Métricas Lean / Kanban

#### Métricas del flujo

- **Lead Time:** tiempo transcurrido desde que el cliente solicita un ítem hasta que lo recibe. Representa la **vista del cliente**.
- **Cycle Time:** tiempo transcurrido desde que se acepta el compromiso hasta el final del proceso para un ítem. Representa la **vista interna del equipo.**
- **Touch Time (tiempo de trabajo activo):** tiempo durante el cual el equipo trabaja. efectivamente sobre un ítem. Excluye las esperas en colas, los bloqueos y los períodos sin actividad sobre él.




**Eficiencia del ciclo de proceso:** porcentaje del tiempo total transcurrido que corresponde a trabajo activo. La fórmula de cálculo es:

**Eficiencia (%) = Touch Time / tiempo total transcurrido × 100**

Ambos tiempos deben corresponder al **mismo tramo del proceso** y expresarse en unidades compatibles. Si se analiza desde el inicio del trabajo hasta su finalización, el denominador es el **Cycle Time**; si se analiza desde la solicitud hasta la entrega, es el **Lead Time**.

**Ejemplo:** un ítem tarda 10 días desde el inicio hasta su finalización, pero solo se trabaja activamente durante 2 días. La eficiencia de ese tramo es **2 / 10 × 100 = 20 %**. El resto corresponde a esperas.

### Métricas orientadas al servicio

Relaciona lo que espera el cliente con lo que el sistema puede entregar y lo que se acuerda:

- **SLE — Expectativa de nivel de servicio:** nivel de servicio que espera el cliente.
- **SLC — Capacidad de nivel de servicio:** nivel de servicio que el sistema puede proporcionar.
- **SLA — Acuerdo de nivel de servicio:** nivel de servicio acordado con el cliente.
- **Umbral de adecuación del servicio:** nivel mínimo por debajo del cual el servicio resulta inaceptable para el cliente.

### Comparación entre enfoques

| Enfoque | Qué se prioriza al medir | Métricas destacadas |
|---|---|---|
| **Tradicional** | Control del proyecto y comparación entre lo planificado y lo real. | Esfuerzo, tiempo, costos y defectos. |
| **Agile** | Trabajo terminado, software funcionando y planificación de iteraciones. | Velocidad, capacidad y RTF. |
| **Lean / Kanban** | Flujo de trabajo, esperas y eficiencia del servicio. | Lead Time, Cycle Time, Touch Time y eficiencia del proceso. |

# U3: Gestión del Software como Producto

---

## 2.1. Introducción a las prácticas continuas

### 2.1.1. Fundamentos

Las **prácticas continuas** son un conjunto de prácticas de desarrollo de software que buscan integrar, verificar y entregar cambios de manera **frecuente, confiable y automatizada**, evitando acumular grandes cantidades de trabajo antes de integrarlo o entregarlo.

Surgen de la necesidad de facilitar la evolución del producto y adaptarse rápidamente a los cambios, manteniendo su calidad y estabilidad.

Las principales prácticas son:

- **Integración Continua (CI):** integrar frecuentemente los cambios de código y verificarlos mediante compilaciones y pruebas automatizadas.
- **Entrega Continua (Continuous Delivery):** mantener el software preparado para ser liberado a producción cuando se decida.
- **Despliegue Continuo (Continuous Deployment):** desplegar automáticamente en producción los cambios que superan las verificaciones establecidas.

#### Objetivos y beneficios

El objetivo principal es **facilitar la entrega frecuente de software de calidad, reduciendo el tiempo y los riesgos asociados a integrar y liberar cambios**.

Sus principales beneficios son:

- **Detección temprana de defectos:** permite identificar problemas rápidamente, evitando que se acumulen.
- **Feedback continuo:** proporciona información frecuente sobre la calidad, estabilidad e integridad del producto.
- **Reducción de riesgos y retrabajos:** los cambios pequeños y frecuentes son más fáciles de integrar, verificar y corregir.
- **Entregas rápidas y confiables:** reduce las actividades manuales y los tiempos necesarios para preparar nuevas versiones.
- **Adaptación al cambio:** facilita incorporar nuevas funcionalidades y correcciones sin comprometer la integridad del producto.

#### Relación con la gestión de configuración y la automatización

La **Gestión de Configuración del Software (SCM)** permite identificar, controlar y realizar seguimiento de los cambios, versiones y componentes que conforman el producto de software, manteniendo su integridad a lo largo del ciclo de vida.

En ambientes ágiles, SCM busca **facilitar la coordinación del equipo y responder a los cambios**, en lugar de centrarse únicamente en mecanismos formales de control.

Las prácticas continuas se apoyan en SCM mediante:

- **Control de versiones y repositorios compartidos:** permiten registrar, integrar y recuperar los cambios realizados por los desarrolladores.
- **Automatización:** permite ejecutar tareas repetitivas, como compilación, pruebas, empaquetado y despliegue, reduciendo errores manuales.
- **Verificación continua:** proporciona feedback sobre la calidad y estabilidad del software después de los cambios.
- **Responsabilidad compartida:** las actividades de configuración e integración forman parte del trabajo cotidiano de todo el equipo.

# 2.2. Integración Continua (CI)

## 2.2.1. Concepto y funcionamiento

La **Integración Continua** es una práctica de desarrollo de software que consiste en integrar frecuentemente los cambios de código realizados por los desarrolladores en un repositorio compartido, verificando que los componentes del sistema funcionen correctamente juntos.

El objetivo es **detectar tempranamente problemas de integración y defectos**, evitando que se acumulen cambios incompatibles que luego sean difíciles y costosos de corregir.

#### Funcionamiento general

1. **Desarrollo:** cada desarrollador realiza pequeños cambios en su espacio de trabajo.
2. **Verificación local:** compila y prueba sus cambios antes de integrarlos.
3. **Integración:** incorpora los cambios al repositorio compartido, sincronizandolos con los del resto del equipo.
4. **Build automatizado:** se compila y construye el software integrado.
5. **Pruebas automatizadas:** se ejecutan pruebas para detectar defectos y comprobar que los cambios no hayan afectado funcionalidades existentes.
6. **Feedback:** se informa el resultado al equipo. Si falla (IC) se prioriza corregir el problema.

La **automatización** permite repetir estas verificaciones rápidamente y con cada integración, reduciendo la dependencia de tareas manuales.

**Beneficios de CI**

- **Detección temprana:** identifica defectos y conflictos de integración rápidamente.
- **Menor retrabajo:** los cambios pequeños y frecuentes facilitan localizar y corregir problemas.
- **Feedback rápido:** el equipo conoce continuamente el estado del software integrado.
- **Mayor estabilidad:** ayuda a mantener una versión compartida y funcional del producto.
- **Facilita las entregas frecuentes:** proporciona una base confiable para Continuous Delivery y Continuous Deployment.

CI busca integrar poco y frecuentemente, verificar automáticamente y corregir rápidamente los problemas, en lugar de acumular cambios y descubrir conflictos al final del desarrollo.

# 2.3. Entrega y Despliegue Continuos

## 2.3.1. Continuous Delivery

La **Entrega Continua (Continuous Delivery)** es una práctica que busca mantener el software en **condiciones de ser liberado a producción en cualquier momento**. Esto lo hace mediante la automatización de las actividades necesarias para preparar y verificar una nueva versión.

Su objetivo es **reducir el tiempo, esfuerzo y riesgo de las entregas**, permitiendo decidir cuándo poner una versión a disposición de los usuarios.

**¿Cómo funciona?**

1. **Integración:** se incorporan los cambios mediante CI.
2. **Verificación:** se realizan compilaciones, pruebas y controles de calidad automatizados.
3. **Preparación:** se genera una versión lista para ser liberada.
4. **Liberación:** una persona autorizada decide cuándo desplegarla en producción.

**Relación con CI:** Continuous Delivery se apoya en Continuous Integration. Mientras **CI** verifica que los cambios se integren correctamente, **CD** busca que el software integrado y verificado permanezca listo para su liberación.

## 2.3.2. Continuous Deployment

El **Despliegue Continuo (Continuous Deployment)** es una práctica que consiste en **desplegar automáticamente en producción** los cambios que superan todas las verificaciones establecidas sin requerir una aprobación manual para cada despliegue.

Su objetivo es **reducir al mínimo el tiempo** entre completar un cambio y ponerlo a disposición de los usuarios, facilitando entregas y feedback frecuentes.

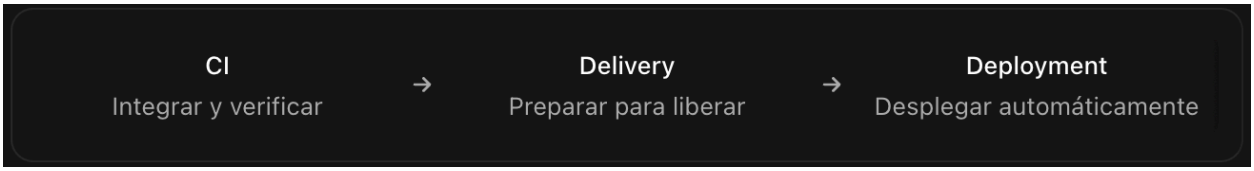
**¿Cómo funciona?**

1. **Integración:** se incorporan los cambios mediante CI.
2. **Verificación:** se ejecutan automáticamente las pruebas y controles de calidad.
3. **Preparación:** se genera la versión lista para producción.
4. **Despliegue automático:** si supera las verificaciones, se publica en producción sin intervención manual.

Para funcionar de manera confiable, requiere **pruebas automatizadas suficientes, monitoreo y mecanismos de recuperación ante fallos**.

**Diferencia entre Continuous Delivery y Continuous Deployment**

| Continuous Delivery | Continuous Deployment |
|---|---|
| Mantiene el software listo para producción. | Publica automáticamente los cambios en producción. |
| La liberación se realiza cuando se decide, pudiendo requerir aprobación manual. | La liberación no requiere aprobación manual por cada cambio. |
| Automatiza la preparación y verificación. | Automatiza también el despliegue a producción. |




## 2.3.3. Estrategias de despliegue

Estas son formas de poner nuevas versiones del software en producción, buscando reducir los riesgos, evitar interrupciones del servicio y facilitar la recuperación ante fallos.

### Canary Deployment

Consiste en **desplegar una nueva versión inicialmente para un pequeño grupo de usuarios**, mientras el resto continúa utilizando la versión anterior.

**Funcionamiento:**

1. Se habilita la nueva versión para un porcentaje reducido de usuarios.

2. Se monitorea su comportamiento y se obtiene feedback real.
3. Si funciona correctamente, se amplía progresivamente al resto.
4. Si aparecen problemas, se detiene el despliegue o se revierte el cambio.

**Ventaja principal:** permite detectar problemas en producción, limitando la cantidad de usuarios afectados.

### Blue/Green Deployment

Consiste en **mantener dos entornos de producción equivalentes**, uno con la versión actual y otro con la nueva versión.

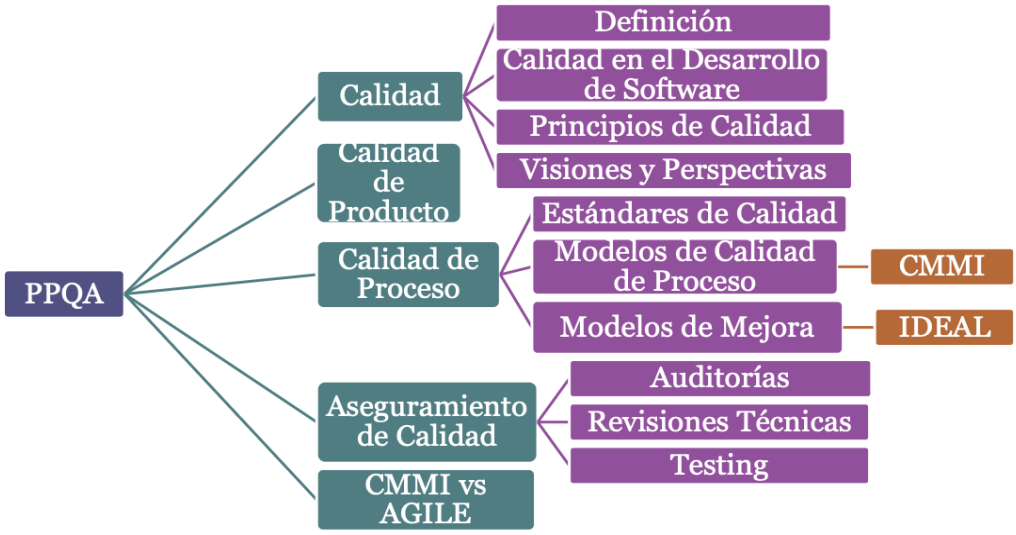
- **Blue:** entorno que actualmente atiende a los usuarios.
- **Green:** entorno donde se despliega y verifica la nueva versión.

Una vez validada la nueva versión, **se redirige el tráfico de Blue a Green**. Si aparecen problemas, puede volver a utilizarse el entorno anterior, siempre que los cambios sean compatibles.

### Diferencias entre ambas estrategias

| Canary Deployment | Blue/Green Deployment |
|---|---|
| Despliega gradualmente a grupos de usuarios. | Mantiene dos entornos de producción. |
| Amplía progresivamente el uso de la nueva versión. | Cambia el tráfico de un entorno al otro. |
| Reduce el impacto de fallos sobre los usuarios. | Facilita cambiar o revertir versiones rápidamente. |

# U4: Aseguramiento de Calidad de Proceso y Producto




## 3.1. Fundamentos de Calidad de Software

### 3.1.1. Conceptos generales de calidad

La calidad es una cualidad subjetiva y relativa a las necesidades y expectativas que tiene una persona. La **calidad de software** es el grado en el que se cumple con estas, ya que consta de una adecuación de un determinado producto o servicio a lo que **necesita o tiene como expectativas** una persona.

Las *expectativas* suelen ser del cliente y del usuario, mientras que las *necesidades* pueden ser de estos, de la gerencia, del equipo de desarrollo, mantenimiento y entre otros interesados.

#### Principios de calidad

Los **principios de calidad** sirven como guías para orientar las decisiones y acciones de una organización, con el objetivo de lograr y mantener la calidad durante el desarrollo de software. En pocas palabras establecen cómo debemos trabajar para **conseguir calidad**:

- La calidad no se inyecta al final (se incorpora durante todo el desarrollo).
- Es un esfuerzo de todos, no solo del equipo de testing.
- No se puede controlar lo que no se mide (necesario el uso de métricas).
- El aseguramiento de calidad debe planificarse, no improvisarse.

### Visiones de calidad

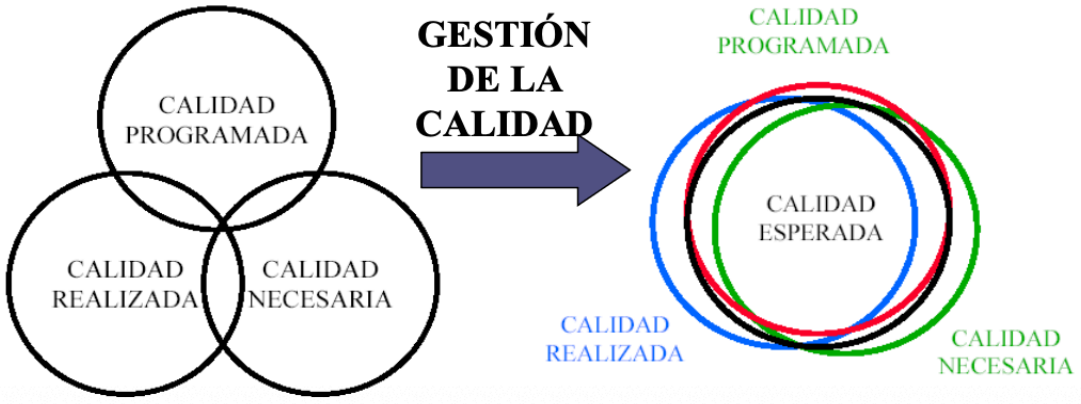
Como dijimos que era relativa, depende de quien la evalúe va ser lo que considere relevante, es decir, **desde qué criterio** algo se considerará con calidad. Existen 5 diferentes visiones:

| Visión | Considera relevante… | Ejemplo (app bancaria) |
|---|---|---|
| Usuario | Que el producto satisfaga sus necesidades y expectativas. | Permite realizar transferencias fácilmente. |
| Producto | Que tenga buenas características o atributos medibles (rendimiento, confiabilidad, entre otros). **RNF**. | Responde rápidamente y tiene pocas fallas. |
| Manufactura | Que el producto cumpla con los requisitos y especificaciones establecidas. | Cumple con los requisitos especificados y pasa las pruebas. |
| Valor | Que exista una buena relación costo-beneficio (calidad obtenida). | Ofrece buenas funcionalidades a un precio razonable. Freemium. |
| Trascendental | Que el producto sea percibido como excelente, aunque resulte difícil medir o definir dicha excelencia. | Es reconocida por su excelencia y calidad en general. Nunca hubo problema. |

### Perspectivas de calidad: necesaria, programada y realizada

La calidad se puede distinguir en tres perspectivas, las cuales tienen que converger para que cumpla con la **calidad esperada**, es decir, con lo que quiere realmente el cliente. La *Gestión de Calidad* es la que se encarga de que estas coincidan lo máximo posible.

La existencia de diferencia entre ellas puede generar **insatisfacción** por parte del cliente, o **desperdicios** que conllevan a costos y retrabajo innecesario, entre otras desventajas.



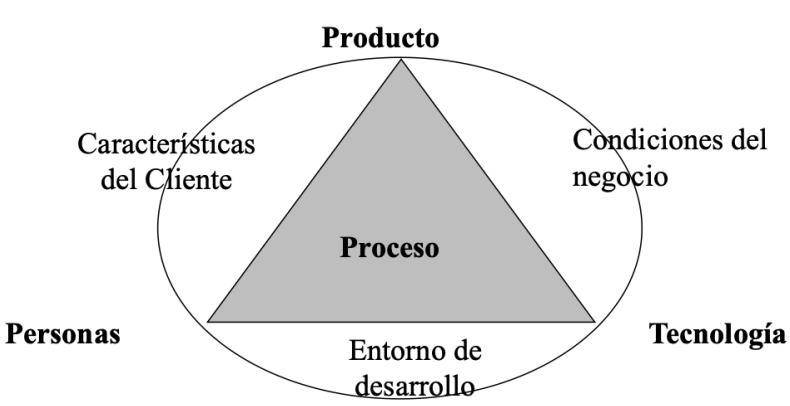
### Calidad de producto vs Calidad de proceso

*Relación breve entre producto-proceso-proyecto: el **producto** es el resultado que se busca obtener; el **proceso** es el conjunto de actividades relacionadas que permiten desarrollarlo o modificarlo (único que tenemos control); el **proyecto** es el esfuerzo temporal que organiza personas, recursos y actividades para alcanzar un objetivo concreto, generalmente mediante la creación o modificación del producto.*

La **calidad del proceso** es el grado en que las actividades, métodos y procedimientos utilizados para desarrollar software están correctamente definidos, se cumplen y se mejoran para favorecer la obtención de productos de calidad.

- Ejemplo: *El equipo establece que el código pase por revisión técnica antes de integrarse.*

El proceso es el único **"factor controlable"** al mejorar la calidad de software y su rendimiento como organización, dado que es aquello que se puede **definir, estandarizar, evaluar y mejorar directamente** para favorecer la calidad de software en general.




Por otro lado, la **calidad del producto** es el grado en que sus características satisfacen las necesidades y expectativas explícitas e implícitas de los usuarios y demás interesados. Las **explícitas** son los RF y los RNF, mientras que las **implícitas** son aquellas necesidades y expectativas que no se expresaron directamente, pero se espera que el producto satisfaga.

- Ejemplo: *App que resulte agradable para el usuario pero presente fallas de seguridad*.

La calidad del producto depende de **múltiples factores** como son: **las personas, tecnología, contexto y proceso**. De estos, el proceso es el principal factor que podemos controlar y mejorar sistemáticamente para favorecer la calidad del producto.

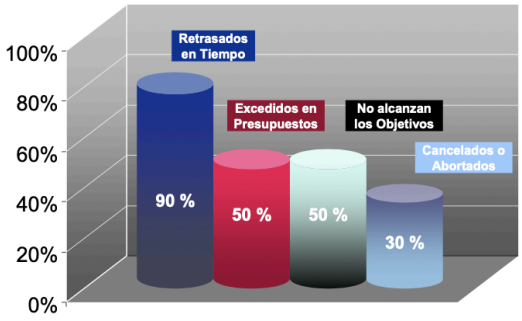
Aunque sea el principal factor, **evaluar o certificar el cumplimiento de un proceso de calidad no va garantizar automáticamente la calidad del producto**, ya que esta también depende de otros factores y de las necesidades de los interesados.

### 3.1.2. Importancia de trabajar con calidad

Es importante dado que permite satisfacer necesidades y expectativas de los interesados, reduciendo costos, retrabajo, y mejorando los procesos de desarrollo. Algo importante es que debemos **mantener la calidad durante todo el proceso**, no intentar inyectarla al final.

Esto es algo relevante debido a que nos permite prevenir y detectar defectos en etapas tempranas, lo cual podemos decir que es algo que nos va ahorrar dolores de cabeza en etapas posteriores del desarrollo. Si enumeramos las ventajas de trabajar con calidad:

- **Satisfacer al cliente** y cumplir con las necesidades y expectativas que tiene
- **Reducir costos**
- **Evitar retrabajo**
- **Reducir retrasos**
- **Mejorar el producto** (menos defectos, mejor cumplimiento de requisitos)

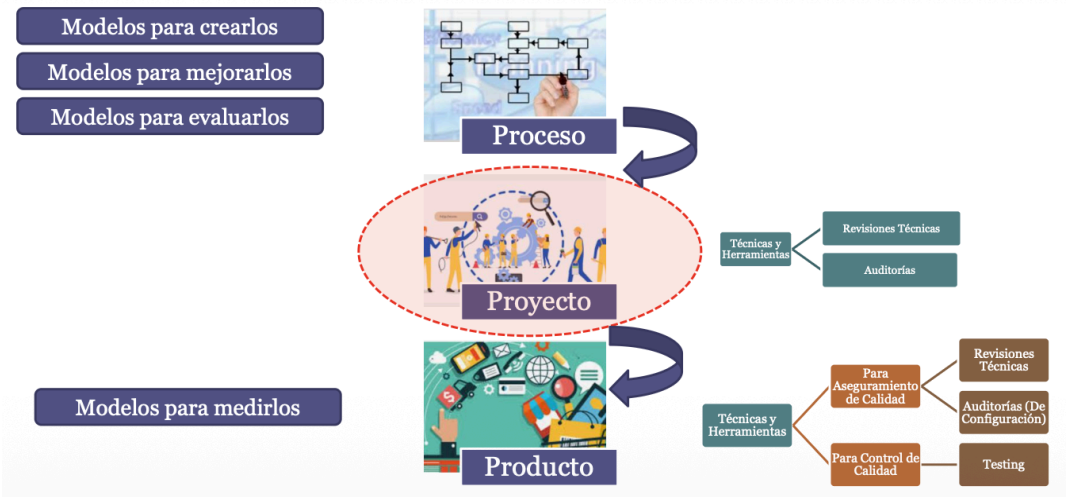



Algunos de los problemas que puede traer no trabajar con calidad es justamente lo contrario por lo cual es importante, es decir, que es posible que se generen retrasos, aumenten los costos a futuro y no se alcance con el cronograma propuesto o con el producto que satisfaga la necesidad real.

### 3.1.3. Gestión y aseguramiento de calidad

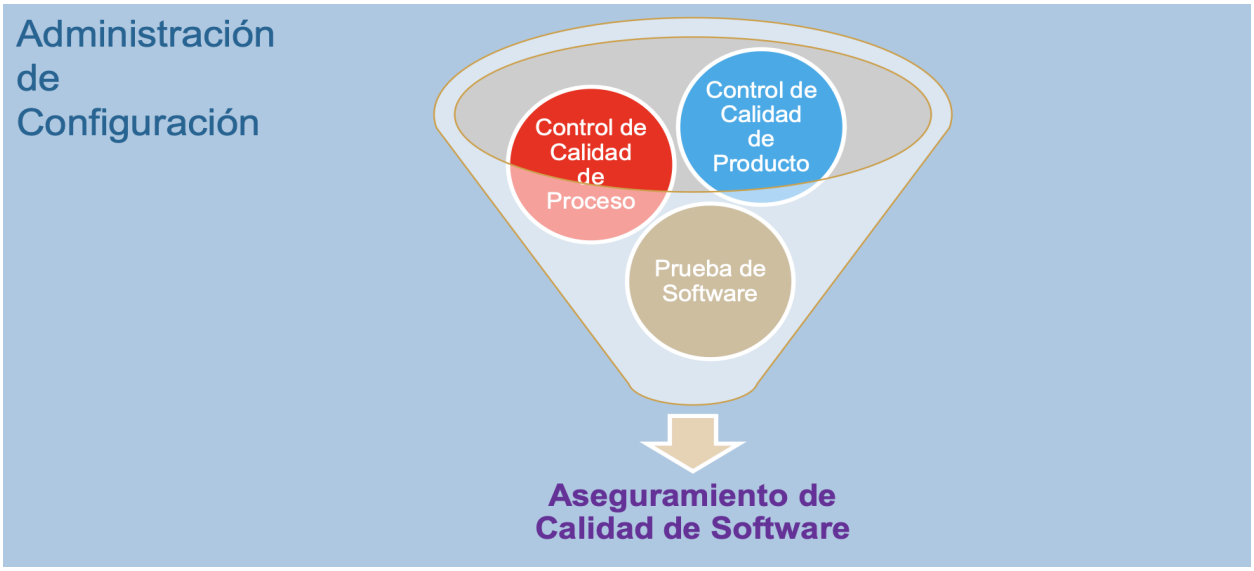
La **Gestión de Calidad** es el conjunto de actividades llevadas a cabo por el *Gerente de Calidad*, destinadas a asegurar que el producto alcance los niveles de calidad esperados por los interesados, tanto en el proceso de desarrollo como en el producto final.

- Ayuda a desarrollar la idea de que la calidad no es responsabilidad de uno solo.
- Conformada por **tres actividades principales**: *QA, QC, Planificación de Calidad*.



### Aseguramiento de Calidad de Software (SQA)

El **Aseguramiento de Calidad** es el conjunto de actividades, el cual **establece los estándares y procedimientos** organizacionales de calidad que deben seguirse durante el desarrollo de software. Su objetivo es favorecer la obtención de productos de calidad mediante procesos definidos, principalmente con un enfoque principalmente preventivo.




### Control de Calidad (QC)

El **Control de Calidad** es el conjunto de actividades, el cual permite **verificar que se respeten los estándares y procedimientos** establecidos. Controla el cumplimiento del proceso y permite evaluar el producto. Enfoque principalmente de detección de incumplimientos y defectos. Contribuye a evaluar la calidad de proceso y de producto mediante:

1) **Revisiones de Calidad:** consisten en examinar procesos, productos de trabajo y su documentación para identificar problemas y verificar el cumplimiento de los criterios de calidad establecidos.
2) **Evaluaciones de Software Automáticas y métricas:** utilizan herramientas y métricas para evaluar determinadas características del software e identificar posibles problemas.

Básicamente, **QC** se diferencia de **QA** debido a que esta se encarga de verificar aquellos estándares y procedimientos de calidad (del proceso como del producto) que el Aseguramiento de Calidad definió previamente.

Cuando hablamos de **estándares de calidad** nos referimos a reglas o criterios establecidos que permiten mantener la calidad durante el desarrollo de software. Están los de *proceso* (cómo realizar las actividades) y los de *producto* (características del producto).

### Planificación de Calidad

La **Planificación de Calidad** es la actividad que consiste en determinar cómo se aplicarán los estándares y reglas del aseguramiento de calidad (QA). Es donde se definen los productos de calidad deseados, junto a los atributos más significativos que debería tener para cumplirla.




Cuando hablamos del **Plan de Aseguramiento de Calidad de Software (PACS)** ahí sí nos referimos al documento de soporte del **Plan de Proyecto** que establece cómo se organizarán y ejecutarán las actividades de aseguramiento de calidad durante el desarrollo del software.

## 3.1.4. Actividades de aseguramiento de calidad

| Actividad | Descripción |
|---|---|
| Definición y evaluación de procesos | Establecer cómo deben realizarse las actividades de desarrollo de software y examinar cómo se están llevando a cabo, verificando el cumplimiento de los estándares e identificando oportunidades de mejora. |
| Planificación y seguimiento | Seleccionar y adaptar estándares y procedimientos de calidad para un proyecto específico, supervisando su correcto cumplimiento. |
| Auditorías | Evaluaciones **objetivas e independientes** que verifican el cumplimiento de los estándares, procedimientos y requisitos establecidos, identificando las desviaciones y oportunidades de mejora. |
| Revisiones técnicas | Actividades de **verificación y validación** que consiste en examinar productos de trabajo (requisitos, diseño, etc.) y detectar defectos en etapas tempranas. |
| Testing | Proceso destructivo donde se somete al software en búsqueda de **defectos**, cuya presencia se asume, y permite hacerlos evidentes. |

# 3.2. Modelos y estándares de calidad

## 3.2.1. Introducción a los modelos de calidad

Los **modelos y estándares** de calidad son referencias que permiten a las organizaciones *definir, evaluar y mejorar la calidad de software*, tanto de los productos desarrollados como de los procesos utilizados para construirlos.

Su objetivo principal es proporcionar criterios, buenas prácticas y procedimientos que ayuden a alcanzar los **niveles de calidad esperados**.

### Modelo de Calidad: de Producto vs de Proceso

Se diferencian principalmente por aquel aspecto de la calidad que evalúan. Mientras que los modelos de producto se enfocan en la calidad de lo *que se construye*, los modelos de proceso se enfocan en *cómo se construye*.

- **Modelo de Calidad de Proceso:** proporciona referencias para evaluar y mejorar los procesos mediante los cuales se desarrolla el software, buscando que sean más definidos, controlados y efectivos. Ejemplos: CMMI y SPICE.
  **Modelo de Calidad de Producto:** define características y atributos que permiten evaluar la calidad del software desarrollado, como seguridad, fiabilidad, rendimiento y mantenibilidad. Ejemplos: ISO/IEC 25010, McCall y Barbacci/SEI.

Se dice que **mejorar la calidad del proceso** favorece la obtención de un producto de mejor calidad, pero **no garantiza automáticamente su calidad final**.

### Modelo vs Norma vs Estándar

- **Modelo:** representación o marco de referencias que organiza características, buenas prácticas o criterios para evaluar y mejorar la calidad. Ejemplo: CMMI.
- **Estándar:** conjunto de criterios, reglas o prácticas establecidas que sirven como referencia común para productos o procesos. Ejemplo: estándares de codificación.
- **Norma:** estándar formalizado y publicado por un organismo de normalización reconocido. Ejemplo: ISO/IEC 25010.

*Los modelos pueden estar definidos dentro de normas, como ocurre con el modelo de calidad de producto de ISO/IEC 25010. Por otro lado, tenemos que una norma es un estándar pero no todo estándar es una norma.*

Por último, debemos distinguir los **modelos de mejora de procesos**, como IDEAL, que no se limitan a definir criterios de evaluación, sino que guían cómo organizar y llevar adelante un proyecto de mejora.

### 3.2.2. Modelos de calidad y mejora de procesos

Los **modelos de calidad y mejora de procesos** proporcionan referencias para definir, evaluar y mejorar los procesos de desarrollo de software de una organización.

Estos modelos pueden establecer buenas prácticas y objetivos, que luego cada organización implementa según su contexto. Es importante diferenciar los modelos que permiten **evaluar el estado de los procesos** de aquellos que guían **cómo implementar las mejoras**.

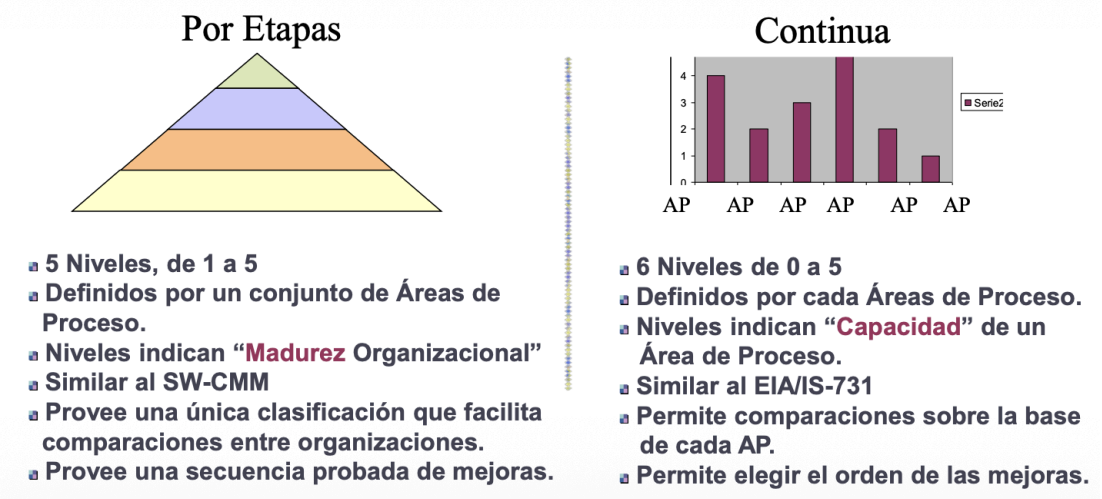
#### CMMI

**CMMI (Capability Maturity Model Integration)** es un modelo de calidad de procesos que proporciona *buenas prácticas* para evaluar y mejorar los procesos de una organización buscando aumentar su *madurez y capacidad*.

Establece **qué objetivos deben cumplir los procesos**, pero no prescribe exactamente cómo implementarlos. Cada organización define y adapta sus métodos y procedimientos al contexto.

Representaciones de CMMI

- **Por etapas:** evalúa la *madurez organizacional* dentro del alcance establecido. Tiene cinco niveles (1-5) y proporciona una secuencia de mejora.
- **Continua:** evalúa la *capacidad de cada área* de proceso individualmente. Tiene seis niveles (0-5) y permite priorizar qué procesos mejorar. El **nivel 0 significa incompleto**, el cual permite identificar áreas que no alcanzan ni siquiera el nivel de ejecución esperado.




La **madurez** representa el grado de desarrollo de un conjunto de procesos, mientras que la **capacidad** representa el grado de desarrollo de un proceso en particular.

Áreas de proceso por nivel y categoría

Un **área de proceso** es un conjunto de prácticas relacionadas que, aplicadas conjuntamente, permiten alcanzar objetivos de mejora en un aspecto del proceso.

| Nivel | Categoría | | | |
|---|---|---|---|---|
| | Administración de Proyectos | Soporte | Administración de Procesos | Ingeniería |
| 5 Optimizado | | ▪ Análisis y Causal y Resolución (CAR) | ▪ Administración de Performance Organizacional (OID) | |
| 4 Cuantitativamente Administrado | ▪ Administración Cuantitativa del Proyecto (QPM) | | ▪ Performance del Proceso Organizacional (OPP) | |
| 3 Definido | ▪ Administración de Riesgos (RSKM)<br>▪Administración Integrada de Proyectos (IPM) | ▪Análisis y Resolución de Decisión (DAR) | ▪ Definición del Proceso Organizacional (OPD)<br>▪ Foco en el Proceso Organizacional (OPF)<br>▪ Capacitación Organizacional (OT) | • **Desarrollo de Requerimientos (RD)**<br>• **Solución Técnica (TD)**<br>• **Integración de Producto (PI)**<br>• **Verificación (VER)**<br>• **Validación (VAL)** |
| 2 Administrado | ▪Administración de Requerimientos (REQM)<br>▪Planificación de Proyectos (PP)<br>▪ Monitoreo y Control de Proyectos (PMC)<br>▪ Administración de Acuerdo con el Proveedor (SAM) | ▪ Aseguramiento de calidad de Proceso y de Producto (PPQA)<br>▪ Administración de Configuración (CM)<br>▪ Medición y Análisis (MA) | | |
| 1 Inicial | **Procesos sin definir o improvisados** | | | |

En el nivel 2, los procesos se planifican, monitorean y controlan a nivel de proyecto. Según **CMMI-DEV v1.3**, comprende siete áreas de proceso, *que son aquellas que integran la materia de Ingeniería y Calidad de Software*.

- **REQM:** Administración de Requerimientos.
- **PP:** Planificación de Proyectos.
- **PMC:** Monitoreo y Control de Proyectos.
- **PPQA:** Aseguramiento de Calidad de Proceso y Producto.
- **CM:** Administración de Configuración (SCM).
- **MA:** Medición y Análisis.
- **SAM:** Administración de Acuerdos con Proveedores.

Para alcanzar un **nivel de madurez** deben satisfacerse las áreas de proceso exigidas para ese nivel y los anteriores, es decir, todos los objetivos que cada área establece. Si una de ellas no cumple dichos objetivos, no se considera alcanzado el nivel 2.

***Ejemplo de SAM:*** *si una empresa terceriza el testing, debe gestionar los acuerdos y verificar el cumplimiento de los compromisos del proveedor (SLA).*

*Las **categorías** permiten agrupar las áreas de proceso por objetivo.*

## SPICE

**SPICE (Software Process Improvement and Capability Determination)** es un marco asociado a la familia ISO/IEC 15504, orientado a **evaluar la capacidad de los procesos** de software. Permite identificar fortalezas y debilidades de los procesos, proporcionando información para orientar su mejora.

**Diferencia con CMMI:** Ambos pueden utilizarse para evaluar y mejorar procesos, pero *CMMI* proporciona un modelo de buenas prácticas con representaciones de madurez y capacidad, mientras que *SPICE* se enfoca en la evaluación de la capacidad de procesos individuales.

La **representación continua de CMMI y SPICE no son lo mismo**, aunque comparten el enfoque de evaluación de procesos. Mientras una apunta a la mejora de áreas de proceso, la otra evalúa e identifica oportunidades de mejora en procesos individuales. Cada uno tiene sus propios criterios de evaluación.

## IDEAL

**IDEAL** es un modelo de mejora de procesos que proporciona una guía para **organizar y ejecutar un plan de proyecto de mejora**. Su objetivo es ayudar a la organización a pasar de "cómo trabaja actualmente" a "cómo quiere trabajar". *IDEAL guía el **proyecto de mejora de procesos**, no el desarrollo de un producto de software*.

**Es cíclico:** se analiza la situación actual, se implementan mejoras y se aprende de los resultados para orientar el siguiente ciclo. Comprende cinco fases:

1. **Iniciar:** reconocer que se necesita mejorar y obtener apoyo.
2. **Diagnosticar:** analizar los procesos actuales y detectar problemas.
3. **Establecer:** definir prioridades y elaborar el plan de acción de mejora.
4. **Actuar:** implementar las mejoras.
5. **Aprender:** evaluar resultados y aprovechar las lecciones aprendidas.

### Relación entre CMMI, SPICE e IDEAL

Los tres modelos se complementan, pero cumplen funciones diferentes:

- **CMMI:** establece buenas prácticas para evaluar y mejorar los procesos.
- **SPICE:** proporciona un marco para evaluar la capacidad de los procesos e identificar las debilidades y necesidades de mejora.
- **IDEAL:** organiza el proyecto para las acciones de mejora.

Mientras que **CMMI y SPICE** sirven como referencia para evaluar procesos y orientar su mejora, **IDEAL** guía cómo planificar, ejecutar y evaluar el proyecto que va permitir implementar esas mejoras.

***Ejemplo:*** *una organización utiliza CMMI para identificar debilidades en sus procesos y posteriormente IDEAL para organizar un proyecto que permita mejorarlos.*

CMMI incluye buenas prácticas para elaborar planes de proyectos de software (PP, nivel 2). IDEAL guía la elaboración y ejecución de un plan de acción destinado a mejorar los procesos de una organización.

| Artefacto | ¿Para qué sirve? | Relación con el modelo |
|---|---|---|
| Plan de Proyecto | Organiza el trabajo necesario para desarrollar un producto: actividades, recursos, estimaciones, cronograma, etc. | CMMI, nivel 2: área PP (Planificación de Proyectos) |
| Plan de Acción de Mejora | Establece qué cambios se realizarán para mejorar los procesos existentes, con responsables, recursos y acciones. | IDEAL, fase Establishing. |
| Resultados de evaluación o diagnóstico | Documentan fortalezas, debilidades y oportunidades de mejora de los procesos. | Evaluaciones con CMMI o SPICE; diagnóstico en IDEAL. |

### 3.2.3. Calidad de procesos en diferentes enfoques

Los distintos enfoques de desarrollo de software buscan mejorar la calidad de los procesos, pero se diferencian en sus prioridades y en la manera de gestionar las mejoras.

#### Enfoque tradicional de mejora de procesos

El enfoque tradicional prioriza la **definición, estandarización, documentación y medición de procesos**, buscando mayor control, estabilidad y predictibilidad en los resultados.

**CMMI** se relaciona con este enfoque porque proporciona buenas prácticas para evaluar y mejorar la capacidad y madurez de los procesos, principalmente a nivel organizacional. Sin embargo, **CMMI no es una metodología tradicional**, sino un modelo de referencia que puede utilizarse con diferentes enfoques de desarrollo.

### Relación entre CMMI y Agile

CMMI y Agile pueden **complementarse**, aunque tienen prioridades y énfasis diferentes:

- **CMMI:** se enfoca en evaluar, controlar y mejorar los procesos organizacionales.
- **Agile:** se enfoca en entregar valor al cliente, adaptarse al cambio y mejorar continuamente mediante retroalimentación.

CMMI define objetivos y buenas prácticas, pero no impone una metodología específica. Por lo tanto, **las prácticas ágiles pueden utilizarse para alcanzar objetivos de CMMI**.

**Por ejemplo:** la planificación de un Sprint, el seguimiento del trabajo y las retrospectivas pueden aportar evidencias del cumplimiento de objetivos relacionados con la planificación, el monitoreo y la mejora de procesos. Sin embargo, debe evaluarse si estas prácticas satisfacen efectivamente los objetivos exigidos por CMMI.

Algunas prácticas de ambos enfoques pueden ser **compatibles**, otras **neutrales** y otras generar **tensiones**, especialmente cuando los controles organizacionales introducen una sobrecarga innecesaria.

### Semejanzas y diferencias entre enfoques

| Aspectos | CMMI | Agile |
|---|---|---|
| Prioridad | Medir y mejorar procesos | Entregar valor y responder al cliente |
| Alcances de mejora | Principalmente organizacional | Principalmente equipo/proyecto |
| Resultados buscados | Predictibilidad y estabilidad | Adaptabilidad y respuesta al cambio |
| Comunicación | Coordinación entre áreas | Comunicación entre personas |
| Gestión del conocimiento | Procesos y activos organizacionales | Colaboración y conocimiento compartido |
| Confianza del cliente | Procesos definidos y gestionados | Software funcionando y participación del cliente |

**Similitudes:**

- Ambos buscan mejorar los resultados y el desempeño de la organización.
- Ambos requieren planificación, seguimiento y reglas de trabajo.
- Ambos se apoyan en la experiencia y la mejora continua.
- Ninguno es aplicable de la misma manera en todos los proyectos.

#### Relación con Lean/Kanban

Desde el **enfoque Lean**, mejorar la calidad de los procesos implica eliminar desperdicios, reducir actividades que no aportan valor y favorecer un flujo de trabajo más eficiente, buscando entregar mayor valor al cliente.

**Kanban** permite aplicar la mejora continua de los procesos partiendo de la forma actual de trabajar, mediante prácticas como visualizar el flujo, limitar el WIP, gestionar el flujo y mejorar colaborativamente.

| |
|---|
| Estas prácticas permiten identificar problemas del proceso, como cuellos de botella, acumulación de trabajo y esperas, para introducir mejoras de manera progresiva. |

**Diferencia principal entre enfoques:**

- **Tradicional:** prioriza procesos definidos, estandarizados y controlados para lograr mayor predictibilidad.
- **Agile:** prioriza la adaptación al cambio, la colaboración y la mejora continua mediante retroalimentación frecuente.
- **Lean/Kanban:** prioriza la eliminación de desperdicios, la optimización del flujo y la mejora evolutiva de los procesos.

Básicamente, **los tres enfoques buscan mejorar la calidad de los procesos**, aunque utilizan diferentes prácticas y prioridades para conseguirlo.

## 3.3. Auditorías y revisiones técnicas

### 3.3.1. Auditorías de software

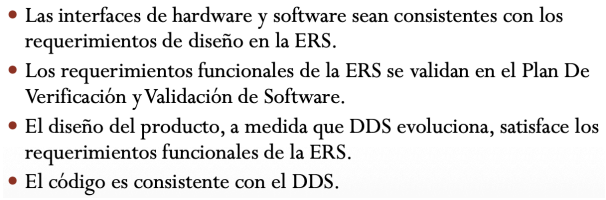
Una **auditoría de software** es una evaluación **objetiva e independiente** de los procesos o productos de software para verificar su cumplimiento con los estándares, procedimientos, especificaciones y lineamientos establecidos. Se basa en evidencias y criterios documentados.

Es necesario llevarlas a cabo debido a que permite asegurar que estamos cumpliendo con las expectativas que tiene el cliente. Además permite identificar áreas de insatisfacción de este, como también oportunidades de mejora.

#### Objetivos y beneficios

- Verificar el cumplimiento de los procesos y estándares definidos.
- Detectar no conformidades; Identificar oportunidades de mejora.
- Brindar información objetiva a la gerencia para tomar decisiones (transparencia).
- Contribuir a mejorar la calidad del producto y la satisfacción del cliente.

### Tipos de auditorías


- Las interfaces de hardware y software sean consistentes con los requerimientos de diseño en la ERS.
- Los requerimientos funcionales de la ERS se validan en el Plan De Verificación y Validación de Software.
- El diseño del producto, a medida que DDS evoluciona, satisface los requerimientos funcionales de la ERS.
- El código es consistente con el DDS.

1. **Auditoría de proyecto (proceso):** verifica que el proyecto cumpla con el proceso de desarrollo establecido y que exista consistencia entre los requerimientos, diseño, código y pruebas *(a lo largo del desarrollo del producto)*. Se planifica según el **PACS** (Plan de Aseguramiento de Calidad de Software).

2. **Auditoría de configuración funcional (producto):** verifica que el software construido cumpla con los requerimientos funcionales especificados en la **ERS** (Especificación de Requerimientos de Software). Comprueba que las funcionalidades requeridas estén implementadas y sean trazables.

3. **Auditoría de configuración física (producto):** verifica que el software construido sea consistente con su documentación técnica y que esta describa correctamente el producto que se entregará.

### Auditoría al Grupo de Aseguramiento de Calidad (GAC)

Evalúa si el propio grupo de calidad cumple con sus responsabilidades, planes y procedimientos establecidos.

Responde a la pregunta: **¿Quién audita al auditor de calidad?**

- Debe realizarla alguien independiente del grupo evaluado, que puede pertenecer a otra área de la organización o ser externo.
- Frecuencia y modalidad dependen de características y necesidades de la organización.
- No debe confundirse con las tres auditorías de proyecto y configuración presentadas.

### Roles y responsabilidades

- **Gerente de SQA:** prepara el plan de auditorías, estima sus costos, asigna recursos y asegura el tratamiento de las no conformidades.
- **Auditor:** realiza la evaluación, recopila evidencias objetivas, identifica hallazgos, elabora el informe y hace seguimiento de las acciones acordadas.
- **Auditado:** proporciona evidencias, participa de la auditoría, responde al informe y propone acciones correctivas para las desviaciones encontradas.

**Independencia del grupo de calidad:** el GAC no debería depender del gerente del proyecto auditado, sino reportar a una instancia superior de la organización, para preservar la objetividad y dar visibilidad de los problemas a la gerencia.

### 3.3.2. Proceso de auditoría




El proceso de auditoría comprende **4 etapas**, aplicables a los tres tipos de auditorías (proyecto, configuración funcional y configuración física). Lo que cambia es el objeto evaluado y los criterios utilizados.

#### a. Preparación y planificación

- Auditor y auditado acuerdan fecha, lugar, alcance, objetivos y participantes.
- Se prepara y revisa la documentación necesaria (preliminar).
- Se establecen los criterios contra los cuales se evaluará el cumplimiento; puede ser que el auditor **prepare el checklist** con los aspectos que verificará durante la auditoría.

#### b. Ejecución

- **Reunión de apertura:** el auditor presenta al equipo auditor, revisa con el auditado el cronograma, alcance y objetivos, explica cómo se realizará la auditoría y aclara dudas.
- **Ejecución propiamente dicha:** el auditor realiza preguntas al auditado, solicita y examina evidencias objetivas y **completa el checklist**, registrando las respuestas y posibles incumplimientos.

#### c. Análisis y reporte de resultados

- El auditor analiza las evidencias y los hallazgos obtenidos.
- **Reunión de cierre:** comunica los resultados preliminares y aclara dudas con el auditado.
- **Reporte final:** documenta los resultados de la auditoría, las desviaciones y las acciones solicitadas.

#### d. Seguimiento

- El auditado propone **plan de acción** para corregir desviaciones, con acciones y fechas.
- El auditor revisa el plan y verifica el cumplimiento de las acciones.
- Una vez verificadas las correcciones, se cierran las desviaciones.

#### Hallazgos de una auditoría (resultados obtenidos al evaluar evidencias)

- **Buenas prácticas:** actividades o procedimientos realizados de manera superior a lo esperado.
- **Desviaciones o no conformidades:** incumplimientos de requisitos, procesos o estándares establecidos. Requieren un *plan de acción correctiva*.

- **Observaciones:** situaciones que, sin constituir necesariamente un incumplimiento demostrado, pueden representar riesgos o futuras desviaciones. No requieren un *plan de acción formal*.

Una observación puede surgir del juicio profesional del auditor, pero debe estar **fundamentada en evidencias objetivas**, no solamente en opiniones personales.

### Técnicas y herramientas

- **Checklist:** lista de preguntas que permite verificar sistemáticamente aspectos definidos. Favorece la consistencia de la auditoría, independientemente de quién la realice dado que el objetivo es *garantizar una evaluación objetiva e imparcial, sin favorecer ni perjudicar al auditado.*
- **Muestreo:** selección de parte representativa de los productos o registros para evaluar.
- **Revisión de registros:** análisis de documentos y evidencias para comprobar el cumplimiento de los procedimientos.
- **Herramientas automatizadas:** ayudan a recopilar información, registrar hallazgos y realizar el seguimiento.

**Importante:** un checklist no garantiza por sí mismo la **independencia del auditor**, sino que ayuda a estandarizar la evaluación.

- **Consistencia:** permite que diferentes auditores evalúen los mismos criterios de manera uniforme y obtengan resultados comparables.
- **Independencia:** implica que el auditor no tenga conflictos de interés con el trabajo que evalúa, garantizando una evaluación objetiva e imparcial.

### Métricas de auditoría

Permiten evaluar el esfuerzo y los resultados de las auditorías. *Cada organización debe seleccionar las métricas apropiadas según sus necesidades.* Algunos ejemplos son:

- **Esfuerzo por auditoría:** cantidad de horas de trabajo dedicadas.
- **Duración:** tiempo transcurrido para realizarla.
- **Cantidad de desviaciones:** incumplimientos identificados.
- **Desviaciones por área de proceso CMMI:** clasificación de incumplimientos según el área de proceso correspondiente.

### 3.3.3. Revisiones técnicas

Una **revisión técnica** es una actividad de **Verificación y Validación (V&V) estática** que consiste en examinar productos de trabajo de software (requerimientos, diseño, código, casos de prueba, etc.) para **detectar defectos lo más temprano posible**, sin necesidad de ejecutar el software.

- *Un producto de trabajo es la salida de cualquier actividad del ciclo de vida de desarrollo.*

Se realiza durante diferentes etapas del ciclo de vida, preferentemente con la **participación** de otras personas, ya que el autor puede pasar por alto sus propios errores.

***Ejemplo:*** *El equipo revisa los requerimientos de un sistema de venta de entradas y se da cuenta que no se especificó cómo evitar la venta de una misma butaca a dos personas. La revisión nos puede permitir detectar este defecto antes de comenzar a programar, evitando costos y retrabajos.*

#### Verificación y Validación (V&V)

Son dos actividades complementarias que se realizan durante todo el ciclo de vida del software:

| Concepto | Responde a… | Enfoque |
|---|---|---|
| Verificación | ¿Estamos construyendo el producto correctamente? | Comprueba que se siguen las especificaciones, estándares y procedimientos establecidos. |
| Validación | ¿Estamos construyendo el producto correcto? | Comprueba que el software satisfaga las necesidades reales del usuario. |

***Ejemplo de app bancaria:*** *no tan solo comprobaremos que la funcionalidad satisfaga todos los requerimientos establecidos para realizar transferencias, sino que el usuario pueda llevarla a cabo y resolver su necesidad real.*

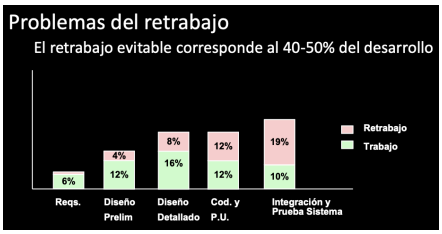
##### Principios de Verificación y Validación

- **La prevención es mejor que la cura:** prevenir defectos resulta más conveniente que corregirlos posteriormente.
- **Evitar es más efectivo que eliminar:** impedir que se **introduzcan** defectos es preferible a detectarlos y corregirlos.
- **La retroalimentación enseña efectivamente:** utilizar los defectos encontrados para aprender y mejorar.
- **Priorizar lo rentable:** concentrar los esfuerzos de V&V donde aporten mayor beneficio.
- **Olvidarse de la perfección:** no es posible garantizar un software full libre de defectos.

- **Enseñar a pescar, en lugar de dar el pescado:** mejorar las capacidades del equipo para prevenir futuros defectos.

## Objetivos, ventajas y desventajas

Objetivos




- Detectar defectos tempranamente y **reducir el retrabajo**.
- Verificar el cumplimiento de requisitos y estándares.
- Mejorar la calidad de los productos de trabajo.

**Ventajas**

- Permiten detectar numerosos defectos antes de ejecutar el software.
- Pueden revisar productos incompletos (documentación, diseños, código, etc.).
- Permiten evaluar distintos atributos de calidad.




**Desventajas**

- Requieren tiempo de preparación, coordinación y capacitación.
- Las inspecciones formales pueden resultar difíciles de incorporar.
- Generan costos iniciales; los beneficios aparecen a medida que el equipo adquiere experiencia.

## Tipos de revisiones técnicas

- En la formal ya hay un proceso, ya hay roles específicos, indicaciones de cómo se pueden combinar dichos roles, duración que se tiene que respetar, se usa el checklist, se documenta, entre otras cosas. Lo contrario a informales.

### Walkthrough (recorrida informal)

Revisión en la que *el autor dirige a los participantes a través de su trabajo*. Es la revisión donde el autor **presenta y explica su trabajo a otros participantes**, quienes hacen preguntas, comentarios y señalan posibles defectos.

1. El autor reúne a otros miembros del equipo.
2. Explica y recorre el producto que desarrolló.
3. Los participantes preguntan y señalan posibles problemas.
4. El autor recibe feedback para mejorar su trabajo.

Cuenta con poca o ninguna preparación, proceso informal, **sin checklist obligatorio** ni mediciones formales. Aunque no es el "óptimo" se recomienda este antes de no tener revisión técnica de ningún tipo.

**Ejemplo:** un programador muestra el diseño de un sistema de pagos. Mientras explica cómo funciona, un compañero pregunta: «¿Qué sucede si el pago es rechazado?». Descubren que ese escenario no está contemplado.

#### Inspección de software (formal)

Es una revisión **planificada, estructurada y sistemática**, que busca detectar y registrar defectos de manera eficiente.

1. Un moderador organiza la inspección y distribuye el material.
2. Los inspectores revisan individualmente el producto antes de la reunión, utilizando criterios definidos.
3. Durante la reunión, se analizan y registran los defectos identificados.
4. El autor corrige los defectos.
5. El moderador verifica las correcciones.

Es un proceso formal, tiene roles definidos, **uso de checklists**, documentación y métricas. También requiere una preparación previa.

**Ejemplo:** un equipo inspecciona el diseño del mismo sistema de pagos. Cada inspector examina el documento utilizando un **checklist** que incluye preguntas sobre manejo de errores, seguridad e interfaces. En la reunión registran los defectos encontrados y luego verifican sus correcciones.

### Proceso de inspección formal y responsabilidades

#### Etapas

1. **Planificación:** el moderador, junto con el autor, selecciona los participantes, asigna roles y organiza la revisión. Distribuye material a inspectores con anticipación (48 hs antes).
2. **Visión general:** se presenta el producto que será inspeccionado, su contexto y el proceso de revisión para que los participantes comprendan qué deben evaluar.
3. **Preparación individual:** cada inspector examina el producto previamente, utilizando criterios o checklists para identificar posibles defectos. Registra los defectos encontrados y el tiempo dedicado a la preparación.
4. **Reunión de inspección:** el equipo analiza los posibles defectos detectados, descarta falsos positivos y registra los hallazgos. Se elabora un **informe de revisión** que indica qué se revisó, quién participó, qué se encontró y las conclusiones. Se decide si el producto se acepta, se rechaza o se acepta provisionalmente.

5. **Corrección (retrabajo):** el autor corrige los defectos identificados y comunica al moderador el esfuerzo empleado y los defectos corregidos.
6. **Seguimiento:** el moderador verifica que las correcciones se hayan realizado correctamente y recopila los datos y métricas de la inspección. Si es necesario, se realiza una nueva revisión.

| Rol | Responsabilidad |
|---|---|
| Autor | •Creador o encargado de mantener el producto que va a ser inspeccionado.<br>•Inicia el proceso asignando un moderador y designa junto al moderador el resto de los roles<br>•Entrega el producto a ser inspeccionado al moderador.<br>•Reporta el tiempo de retrabajo y el nro. total de defectos al moderador. |
| Moderador | •Planifica y lidera la revisión.<br>•Trabaja junto al autor para seleccionar el resto de los roles.<br>•Entrega el producto a inspeccionar a los inspectores con tiempo (48hs) antes de la reunión.<br>•Coordina la reunión asegurándose que no hay conductas inapropiadas<br>• Hacer seguimiento de los defectos reportados. |
| Lector | Lee el producto a ser inspeccionado. |
| Anotador | Registra los hallazgos de la revisión |
| Inspector | Examina el producto antes de la reunión para encontrar defectos. Registra sus tiempos de preparación. |

Técnicas y herramientas

1. **Checklist (lista de verificación)**

Es una **lista de preguntas o criterios concretos** que guía al revisor para examinar un producto de trabajo (arquitectura, requerimientos, código, casos de prueba, etc.) e **identificar posibles defectos, errores u omisiones**.

- Se elabora según el **tipo de producto que se revisará**, sus requisitos y los defectos que se quieren detectar.
- Preguntas claras, específicas y verificables, permitiendo responder **Sí / No / No aplica**.
- Durante la revisión, los inspectores utilizan el checklist para comprobar cada criterio y registrar posibles defectos.
- Ayuda a **estandarizar las revisiones**, evitar omisiones y facilitar la detección sistemática de defectos.

**¿Cómo diseñar un checklist en el parcial?**

- **Identificar el producto:** determinar qué se pide revisar (arquitectura, código, casos de prueba, etc.).
- **Identificar qué evaluar:** pensar qué defectos podrían existir en ese producto (información faltante, inconsistencias, errores, etc.).
- **Formular preguntas verificables:** redactar preguntas específicas que permitan comprobar si existe algún problema.

- **Cubrir distintos aspectos:** evitar preguntas repetidas y asegurar que cada una evalúe algo diferente.

Las preguntas deben evaluar el producto solicitado **sin ejecutar el software**, ya que se trata de una revisión técnica estática. Estas preguntas no pretenden comprobar si el software funciona. Pretenden **encontrar posibles problemas en el diseño de…** *la arquitectura, los casos de prueba documentados, código fuente, plan de proyecto, entre otras cosas*.

**Por ejemplo**, ante la pregunta «¿Están documentadas las interfaces entre componentes?», si la respuesta es No, se detecta una posible omisión en la documentación arquitectónica.

**Reglas fundamentales para formular buenas preguntas**

| | |
|---|---|
| **Específicas** | Preguntar por un aspecto concreto, no simplemente «¿Está bien hecho?». |
| **Verificables** | Deben poder responderse examinando el producto o sus documentos. |
| **Relacionadas con el producto** | No evaluar código cuando la consigna solicita revisar arquitectura. |
| **Sin ejecución** | Una revisión técnica es estática; no se necesita ejecutar el sistema para responder. |
| **No redundantes** | Cada pregunta debería evaluar un aspecto diferente. |

Estos son ejemplos orientativos. No todos los criterios corresponden a todos los productos; deben adaptarse al producto, sus requisitos y el contexto de la consigna.

2. **Métricas de inspección**

Permiten medir el esfuerzo y los resultados de las inspecciones para evaluar su efectividad y mejorar el proceso.

- **Densidad de defectos:** cantidad de defectos encontrados / tamaño producto revisado.
- **Esfuerzo de inspección:** suma del esfuerzo de planificación, preparación, reunión y retrabajo.
- **Esfuerzo por defecto:** esfuerzo total de inspección / cantidad de defectos encontrados.

### 3.3.4. Comparación de actividades de calidad

Las **auditorías, revisiones técnicas y pruebas de software (testing)** son actividades complementarias que contribuyen a mejorar la calidad del software, pero tienen diferentes objetivos.

Las tres actividades se complementan, pero ninguna reemplaza a las demás. La calidad debe trabajarse durante todo el proceso de desarrollo, no solamente mediante pruebas al finalizar.

| Aspecto | Auditorías | Revisiones técnicas | Testing |
|---|---|---|---|
| **Objetivo** | Verificar el cumplimiento de procesos, estándares y requisitos establecidos. | Detectar defectos tempranamente en los productos de trabajo. | Detectar defectos y fallas evaluando el comportamiento del software. |
| **¿Qué evalúan?** | Procesos o productos, según el tipo de auditoría. | Productos de trabajo: requisitos, diseño, código, casos de prueba, etc. | Componentes, integraciones o el sistema completo. |
| **¿Cuándo?** | Según lo planificado en el PACS; durante el proyecto o antes de la entrega, según el tipo. | Durante las distintas etapas del desarrollo, incluso sobre productos incompletos. | Durante el desarrollo, cuando existen componentes ejecutables, y en niveles posteriores del sistema. |
| **¿Quiénes participan?** | Auditor, auditado y responsable de SQA. | Autor y revisores; en inspecciones formales, moderador, inspector, lector y anotador. | Desarrolladores, testers y usuarios, según el nivel de prueba. |
| **¿Requiere ejecutar el software?** | No necesariamente. | No. Es una actividad estática. | Sí, en las pruebas dinámicas. |

### Diferencias fundamentales

- **Auditoría vs. Revisión técnica:** la auditoría comprueba el cumplimiento de criterios establecidos, mientras que la revisión técnica busca detectar defectos en el producto de trabajo.
- **Revisión técnica vs. Testing:** la revisión técnica examina productos sin ejecutar el software, mientras que las pruebas dinámicas ejecutan el software para detectar fallas y evaluar su comportamiento.

**Ejemplo (desarrollo de un sistema bancario)**

- **Auditoría:** comprobar que el equipo haya realizado las revisiones de código exigidas por el proceso.
- **Revisión técnica:** examinar el código de transferencias para detectar errores lógicos o condiciones no contempladas.
- **Testing:** ejecutar una transferencia con saldo insuficiente y comprobar que el sistema la rechace correctamente.

# 3.4. Testing de Software

## 3.4.1. Fundamentos de testing

El **testing de software** es un proceso destructivo donde se somete a un componente de software a condiciones específicas con el fin de determinar si es válido y hace lo que fue destinado a hacer (requerimientos). Consiste en evaluar un producto, **buscar defectos, cuya presencia se asume, y hacerlos evidentes**.

Como objetivos principales el testing tiene **detectar defectos** (comportamientos inesperados o incorrectos) y **evaluar el cumplimiento de requisitos** (comprobar que el software se comporte según lo especificado y aporte confianza).

- Un testing es exitoso **cuando encuentra defectos**.
- El software siempre va a tener defectos, sin importar si las pruebas pasan.
- El testing **no asegura calidad** ni da garantías de que no hay ningún error.
- Detectar errores en etapas tempranas es siempre menos costoso.
- Se puede empezar antes de la codificación (requerimientos + casos de prueba).

### Principios de testing (7)

1. **Las pruebas muestran la presencia de defectos:** no demuestran que el software esté libre de ellos.
2. **El testing exhaustivo es imposible:** no se pueden probar todas las combinaciones posibles.
3. **Testing temprano:** comenzar las pruebas cuanto antes reduce costos y retrabajo.
4. **Agrupamiento de defectos:** suelen concentrarse en determinados componentes.
5. **Paradoja del pesticida:** repetir siempre las mismas pruebas puede dejar de descubrir nuevos defectos; deben revisarse y actualizarse.
6. **Testing dependiente del contexto:** las pruebas varían según las características y los riesgos del sistema.
7. **Falacia de ausencia de errores:** un sistema sin defectos detectados puede seguir siendo inútil si no satisface las necesidades del usuario.

### Error, defecto y falla

- **Error:** problema o equivocación humana detectada y corregida en la misma etapa donde se originó.
- **Defecto:** problema introducido en un producto de trabajo (requisito, diseño o código) que pasa inadvertido a una etapa posterior.
- **Falla:** comportamiento incorrecto observable cuando se ejecuta el software.

**Ejemplo:** desarrollador programa incorrectamente una validación (error humano). El código incorrecto pasa a testing (defecto). Al ingresar un valor válido, el sistema lo rechaza (falla).

### Severidad vs Prioridad

La **severidad** indica cuánto afecta el defecto al funcionamiento del sistema. El equipo necesita acordar qué significa cada nivel para clasificar los defectos de manera consistente:

- **Bloqueante:** impide usar el sistema o continuar las pruebas.
- **Crítico:** el sistema funciona, pero una función esencial no permite cumplir su objetivo. Ejemplo: se puede armar un pedido, pero no finalizar la compra.
- **Mayor:** una función importante falla, aunque existe una alternativa para seguir trabajando. Ejemplo: no funciona la búsqueda, pero se puede encontrar el producto navegando por categorías.
- **Menor:** falla una función secundaria o un caso poco frecuente, con impacto limitado. Ejemplo: un filtro que devuelve resultados incorrectos.
- **Cosmético:** problema de presentación (no impide usar la función). Ejemplo: un ícono desalineado.

La **prioridad** indica cuánta urgencia tiene la resolución de ese defecto a corregirse:

- **Urgente:** requiere atención inmediata, por ejemplo, porque afecta una actividad esencial o una entrega próxima.
- **Alta:** debe corregirse pronto y ubicarse entre las primeras tareas pendientes.
- **Media:** conviene corregirlo, pero puede planificarse junto con otras tareas.
- **Baja:** puede posponerse porque hay asuntos más urgentes. Incluso un defecto grave podría tener prioridad baja si, por ejemplo, afecta una función que todavía no se utiliza.

Normalmente, la **severidad** la propone el *equipo de testing* al detectar el defecto. La **prioridad** la define el *PO* o *Líder de proyecto* según la necesidad que tiene en el proyecto. Un defecto puede tener **baja severidad y alta prioridad**.

### ¿Cuánto testing es suficiente?

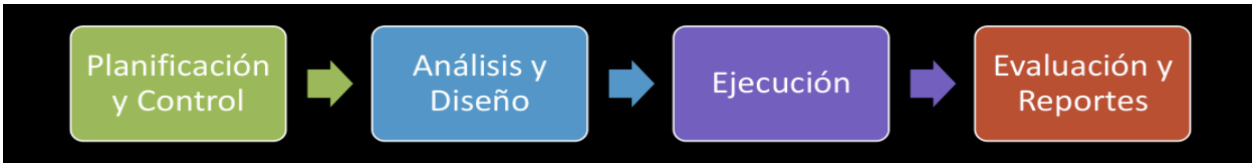
El testing exhaustivo es prácticamente imposible por la cantidad de combinaciones de datos, funcionalidades y condiciones posibles. Por eso, se busca **maximizar la detección de defectos con el tiempo y los recursos disponibles**, priorizando según el riesgo. Para decidir esto:

- **Riesgos:** funcionalidades críticas o con mayor probabilidad e impacto de fallas.
- **Costos y recursos:** tiempo, presupuesto y esfuerzo disponible.
- **Criterios de finalización:** condiciones previamente establecidas para decidir cuándo detener las pruebas.

Estos criterios pueden incluir un porcentaje de pruebas exitosas, la cobertura alcanzada y la ausencia de defectos de determinada severidad.

### Proceso de pruebas

Conj. de actividades planificadas para organizar, diseñar, ejecutar y evaluar pruebas de SW.




1. **Planificación y control**
    - Se definen objetivos, riesgos, estrategia, recursos y criterios de finalización.
    - Se elabora el **Plan de Pruebas (Test Plan)**.
    - Se monitorean resultados, cobertura y avance para tomar decisiones.
2. **Análisis y diseño**
    - Se analizan los requerimientos y su posibilidad de ser probados.
    - Se identifican las condiciones y los datos de prueba.
    - Se diseñan y priorizan los **casos de prueba**, definiendo resultados esperados.
    - Se diseña el ambiente de prueba.
3. **Ejecución**
    - Se prepara el ambiente y los datos necesarios.
    - Se ejecutan los casos de prueba sobre una versión identificada del software.
    - Se comparan los resultados obtenidos con los esperados.
    - Se registran los resultados y los defectos encontrados.
4. **Evaluación y reportes**
    - Se evalúa el cumplimiento de los criterios de finalización.
    - Se reportan resultados, cobertura, defectos pendientes y riesgos.
    - Se verifica la corrección de defectos y se registran las lecciones aprendidas.

La planificación y el diseño pueden realizarse **antes de disponer de una versión ejecutable**. En Agile, estas actividades acompañan continuamente el desarrollo de cada incremento.

### Ciclo de Prueba

Un **ciclo de prueba** comprende la ejecución del conjunto de casos de prueba establecidos sobre una misma versión del software.

- Se ejecutan los casos de prueba y se registran los defectos.
- Los desarrolladores corrigen los defectos y generan una nueva versión.
- Se inicia otro ciclo para comprobar las correcciones y verificar que no se hayan introducido nuevos defectos.

La **prueba de regresión** consiste en volver a ejecutar pruebas anteriores después de una modificación para comprobar que las funcionalidades que antes funcionaban no se hayan visto afectadas.

### Ambientes de testing

| Ambiente | Propósito |
|---|---|
| **Desarrollo** | Construcción del software y pruebas unitarias e integración. |
| **Prueba (Testing)** | Ejecución controlada de pruebas, principalmente de sistema. |
| **Preproducción** | Simular condiciones de producción y realizar pruebas de aceptación. |
| **Producción** | Uso real del sistema por parte de los usuarios. |

Se recomienda separar los ambientes para evitar interferencias, controlar versiones y datos, y reproducir defectos. Se consideran **3 ambientes como mínimo** (desarrollo, prueba y producción) y **4 como situación ideal** (más caro), incorporando preproducción.

### Plan de pruebas y Casos de prueba (artefactos fundamentales)

- **Plan de Pruebas (Test Plan):** documento que establece qué se va a probar, cómo, cuándo, con qué recursos, responsables y criterios de finalización.
- **Caso de Prueba (Test Case):** descripción concreta y reproducible de una situación que se quiere comprobar, indicando sus condiciones, datos, acciones y resultados esperados.

#### Elementos esenciales del caso de prueba

1. **Objetivo:** característica o comportamiento que se quiere comprobar.
2. **Datos de entrada y ambiente:** valores y condiciones necesarias para ejecutar la prueba.
3. **Resultado esperado:** comportamiento que debería producir el sistema.

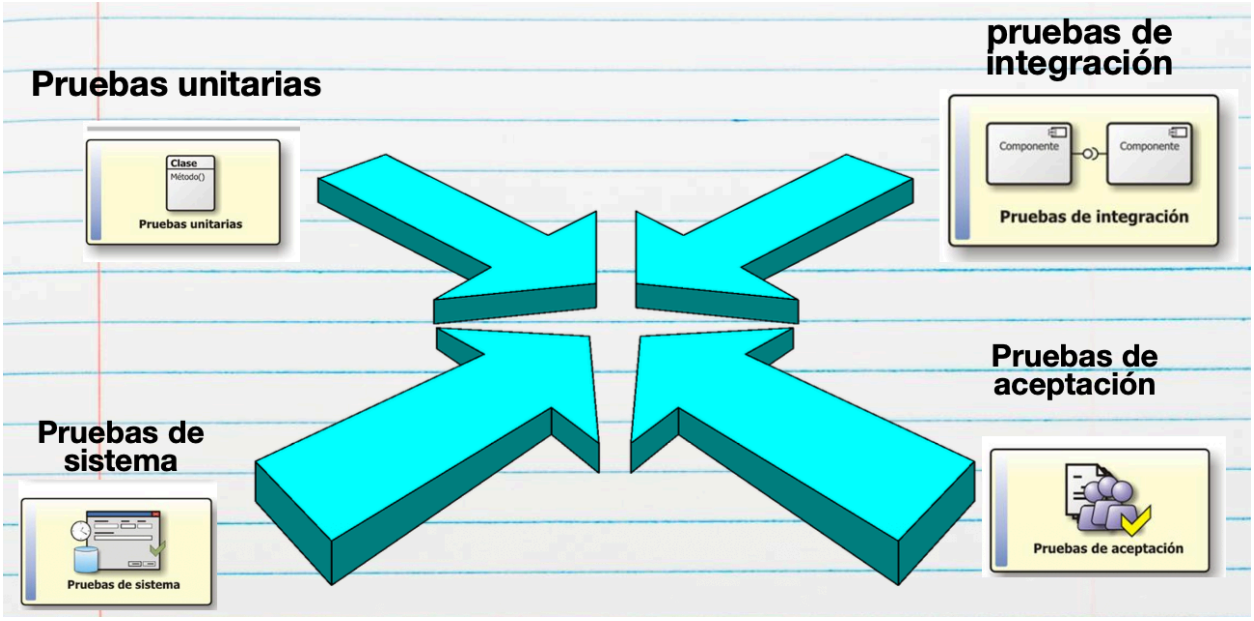
En la práctica, se documentan también ID, nombre, prioridad, precondiciones y pasos.

**La reproducibilidad es fundamental:** un caso debe permitir que otra persona ejecute la misma prueba y pueda investigar el defecto encontrado.

## 3.4.2. Niveles y tipos de testing

### Niveles de prueba

Los niveles de testing representan el **grado de integración o granularidad del software que se prueba**. Se comienza con componentes individuales y se avanza hacia el sistema completo y su aceptación por el usuario.




#### Pruebas unitarias (componentes)

Verifican el funcionamiento de componentes individuales y aislados (func., clases, módulos).

- Generalmente realizadas por los desarrolladores, con acceso al código.
- Pueden automatizarse utilizando herramientas como JUnit.
- ***Ejemplo:*** comprobar que una función calcule correctamente un descuento.

#### Pruebas de integración (interfaces)

Verifican que dos o más componentes que funcionan individualmente también funcionen correctamente al interactuar.

- Detectan problemas de comunicación, interfaces e intercambio de datos.
- La integración debe ser incremental (Top-Down, Bottom-Up o combinada).

- ***Ejemplo:*** comprobar que el módulo de compras envíe correctamente el importe al módulo de pagos.

### Pruebas de sistema

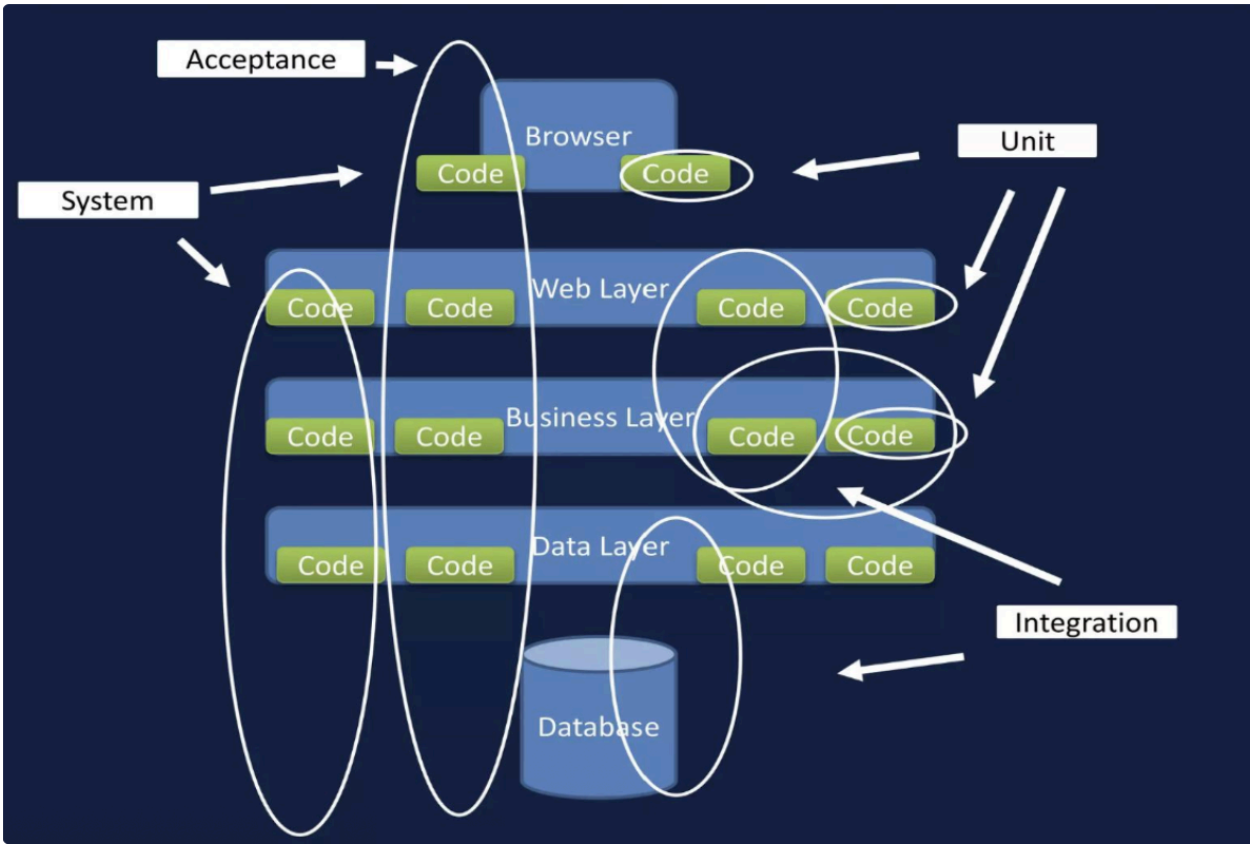
Verifican el funcionamiento del **sistema completo e integrado**, comprobando que cumpla los requisitos funcionales y no funcionales.

- Se realizan sobre una versión integrada del producto.
- Se recomienda que las ejecute un equipo independiente del desarrollo.
- El ambiente debe representar al de producción tanto como sea posible.
- ***Ejemplo:*** comprobar el proceso completo de seleccionar productos, comprarlos y pagar.

### Pruebas de aceptación

Verifican si el producto satisface las **necesidades y expectativas del usuario**, para determinar si puede aceptarse.

- Las realiza o valida el usuario o cliente.
- Su objetivo principal es generar confianza y comprobar la adecuación del producto, más que buscar defectos.
- Pueden ser **alfa** (en un ambiente controlado) o **beta** (en condiciones reales de uso).
- ***Ejemplo:*** el usuario comprueba que el proceso de compra satisface sus necesidades.


Acceptance
System
Browser
Code
Code
Unit
Web Layer
Code
Code
Code
Code
Business Layer
Code
Code
Code
Code
Data Layer
Code
Code
Code
Code
Integration
Database

### Tipos de prueba

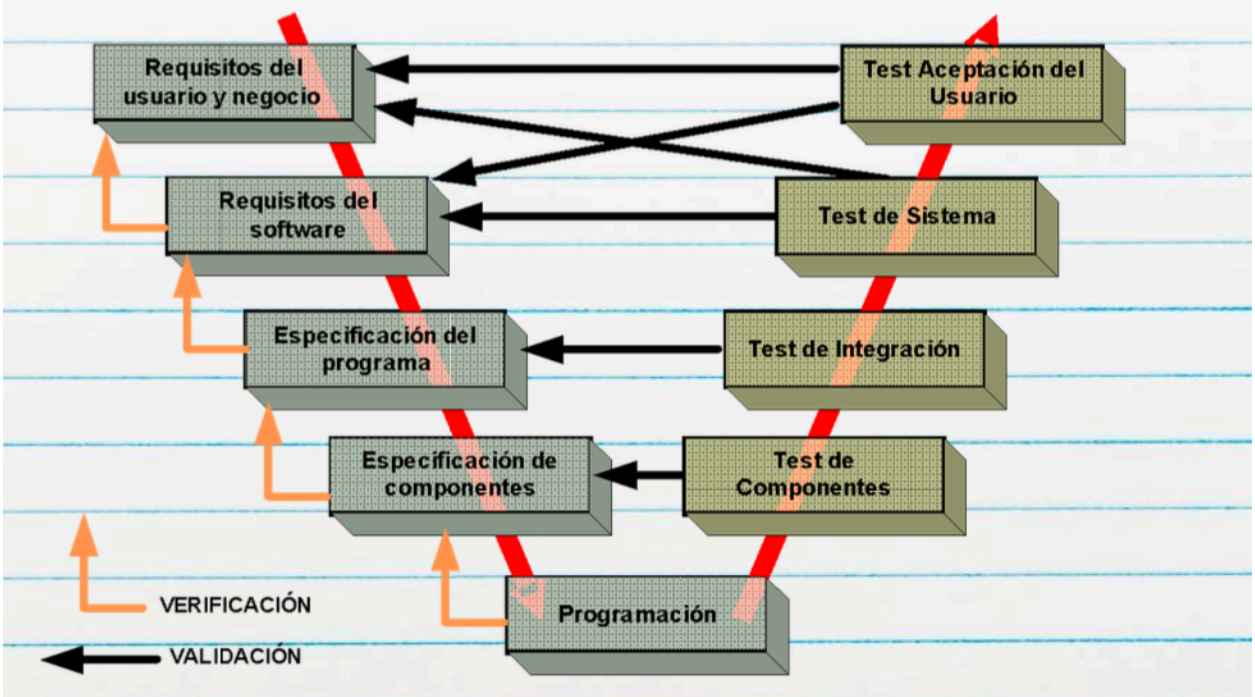
Los tipos de testing permiten evaluar diferentes características del software. **Un mismo nivel de prueba puede incluir distintos tipos de pruebas.**

1. **Pruebas funcionales:** verifican **qué hace el software**, comprobando que sus funcionalidades y procesos de negocio cumplan los requisitos establecidos.
    - Se basan en requerimientos funcionales y comportamientos esperados.
    - *Ejemplo:* comprobar que el sistema aplique correctamente un descuento.
2. **Pruebas no funcionales:** evalúan **cómo funciona el software**, comprobando atributos de calidad y requisitos no funcionales.
   Principales pruebas no funcionales:
    - **Performance (rendimiento):** evalúa tiempos de respuesta y uso de recursos.
    - **Carga:** comprueba el comportamiento bajo la carga de trabajo prevista.
    - **Estrés:** somete el sistema a condiciones superiores a las previstas para observar sus límites y recuperación.
    - **Usabilidad:** evalúa facilidad de uso e interacción.
    - **Fiabilidad:** comprueba el comportamiento consistente y confiable del sistema.
    - **Portabilidad:** verifica su funcionamiento en distintos entornos acordados.
    - **Mantenibilidad:** evalúa la facilidad para modificar y mantener el software.
    - **Seguridad:** comprueba la protección de datos y el control de accesos.
3. **Pruebas de regresión:** consisten en **volver a ejecutar pruebas previamente realizadas después de modificar el software**, para comprobar que los cambios no hayan afectado funcionalidades que antes funcionaban correctamente.
    - Se realizan después de correcciones, cambios o incorporación de funcionalidades.
    - Pueden abarcar todos los casos de prueba o un subconjunto seleccionado según el riesgo.
    - Son especialmente útiles cuando están automatizadas.
    - *Ejemplo:* después de corregir el pago con tarjeta, volver a probar otros medios de pago.
4. **Smoke Test (prueba de humo):** primera comprobación rápida de una versión del sistema para detectar fallas graves que impidan continuar con pruebas más completas.
    - Verifica funcionalidades críticas y el funcionamiento básico.
    - Si falla, se devuelve la versión para su corrección antes de continuar.
    - *Ejemplo:* comprobar que la aplicación inicie, permita iniciar sesión y acceda a sus funciones principales.

**Diferencia importante:** el Smoke Test verifica si una versión está en condiciones de seguir probándose; la regresión verifica que los cambios no hayan afectado funcionalidades existentes.

### Modelo en V (y relación con niveles de prueba)

El **Modelo en V** representa la relación entre las actividades de desarrollo y los niveles de testing en un enfoque tradicional.




- **Lado izquierdo:** se definen y especifican los requisitos, el diseño y los componentes del software.
- **Parte inferior:** se realiza la programación.
- **Lado derecho:** se ejecutan las pruebas, comenzando por las unitarias y avanzando hasta las de aceptación.

Cada nivel de prueba se relaciona con una actividad de definición o diseño:

| Actividad de desarrollo | Nivel de prueba relacionado |
|---|---|
| Requisitos del usuario y negocio | Pruebas de aceptación |
| Requisitos del software | Pruebas de sistema |
| Especificación del programa | Pruebas de integración |
| Especificación de componentes | Pruebas unitarias |

**Idea fundamental:** las pruebas deben planificarse desde las primeras etapas del desarrollo, y no solamente cuando termina la codificación.

En **Agile**, estos mismos niveles siguen existiendo, pero las pruebas se realizan continuamente sobre incrementos pequeños, en lugar de esperar a disponer del producto completo.

### 3.4.3. Estrategias y métodos de testing

Las **estrategias de testing** indican en qué información nos basamos para diseñar las pruebas. Los **métodos o técnicas** permiten seleccionar casos de prueba efectivos, buscando detectar la mayor cantidad posible de defectos con tiempo y recursos limitados.

Existen dos estrategias principales: **caja blanca y caja negra**. No son excluyentes, sino complementarias.

#### Caja blanca

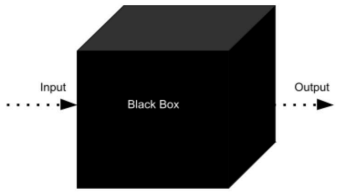
Es una estrategia que diseña las pruebas a partir del **conocimiento de la estructura interna y lógica del código fuente**.

- Requiere conocer cómo está implementado el software.
- Permite evaluar qué instrucciones, condiciones y caminos del código se ejecutaron.
- Se utiliza frecuentemente en pruebas unitarias.

Sus principales técnicas se basan en criterios de cobertura:

- **Sentencias:** ejecutar las instrucciones del código.
- **Decisiones o ramas:** probar los posibles resultados de cada decisión (verdadero y falso).
- **Condiciones:** evaluar individualmente las condiciones lógicas.
- **Caminos:** recorrer diferentes secuencias de ejecución del programa.

#### Caja negra




Es una estrategia que diseña las pruebas a partir de las **especificaciones y el comportamiento esperado del software**, sin necesitar conocer su implementación interna.

- Se analizan las entradas, salidas y reglas de negocio.
- Se comprueba que el sistema responda según los requerimientos.
- Se utiliza frecuentemente en pruebas funcionales y de aceptación.

Los métodos de caja negra se dividen en dos grupos:

1. **Técnicas basadas en especificaciones**

Utilizan los requerimientos del software para identificar condiciones y comportamientos que deben comprobarse.

- **Partición de equivalencias:** divide los posibles valores de entrada y salida en **clases de equivalencia**, cuyos elementos deberían producir comportamientos equivalentes.
    - Se identifican clases válidas e inválidas.
    - Se seleccionan valores representativos de cada clase para diseñar los casos.
    - Permite reducir la cantidad de pruebas sin tener que comprobar todos los valores posibles.
- *Ejemplo:* si se aceptan edades entre 18 y 65 años, las clases de entrada son: menores de 18 (inválida), de 18 a 65 (válida) y mayores de 65 (inválida).

- **Análisis de valores límite:** complementa la partición de equivalencias seleccionando valores **en los extremos de las clases y alrededor de sus fronteras**, donde suelen encontrarse defectos.
  *Ejemplo:* para el intervalo válido de edades entre 18 y 65 años inclusive, se prueban: 17, 18, 19, 64, 65 y 66.

- **Tablas de decisión:** representan combinaciones de condiciones y las acciones o resultados esperados para cada combinación.
    - Son útiles cuando una funcionalidad depende de varias reglas de negocio.
    - *Ejemplo:* permitir una compra solamente cuando el usuario está autenticado y existe stock.

- **Pruebas por pares (Pairwise):** seleccionan casos de manera que cada combinación posible de valores de dos parámetros aparezca al menos una vez.
    - Reducen la cantidad de combinaciones respecto de probarlas todas.
    - *Ejemplo:* probar combinaciones de navegador, sistema operativo y tipo de usuario.

- **Grafos causa-efecto:** representan mediante relaciones lógicas cómo las condiciones de entrada (**causas**) producen determinados resultados (**efectos**).
    - Permiten identificar combinaciones de condiciones y derivar casos de prueba mediante tablas de decisión.

2. **Técnicas basadas en la experiencia**

Utilizan el conocimiento y la experiencia del tester para identificar posibles defectos.

- **Adivinanza de defectos (Error Guessing):** consiste en anticipar errores probables basándose en defectos frecuentes o experiencias anteriores.
    - *Ejemplo:* ingresar caracteres especiales en un campo numérico.

- **Testing exploratorio:** el tester aprende sobre el software mientras diseña y ejecuta pruebas, adaptándolas según los resultados obtenidos.
    - No requiere definir todos los casos de prueba por adelantado.
    - Puede realizarse mediante sesiones con objetivos y registro de hallazgos.

**Diferencias entre caja blanca y caja negra**

| Aspecto | Caja blanca | Caja negra |
|---|---|---|
| **Se basa en** | Estructura interna del código. | Especificaciones y comportamiento esperado. |
| **Conocimiento del código** | Necesario. | No necesario. |
| **Qué comprueba** | Lógica, decisiones y caminos internos. | Entradas, salidas y reglas de negocio. |
| **Técnicas principales** | Cobertura de sentencias, decisiones y condiciones. | Equivalencias, límites y técnicas basadas en experiencia. |
| **Uso frecuente** | Pruebas unitarias. | Pruebas funcionales y de aceptación. |

**Conclusión:** ningún método permite detectar todos los defectos. Por eso, se recomienda **combinar diferentes técnicas de prueba**, aprovechando sus fortalezas y compensando sus limitaciones.

### 3.4.4. Testing en ambientes Agile

### Testing Tradicional vs. testing Agile

La principal diferencia está en **cuándo y cómo se realiza el testing**, y en quiénes participan de la calidad del producto.

| Aspecto | Tradicional (cascada) | Agile |
|---|---|---|
| Momento | La ejecución de pruebas se concentra principalmente después de la implementación. | Las pruebas acompañan continuamente el desarrollo. |
| Responsabilidad | Suelen existir roles o equipos especializados. | La calidad es responsabilidad de todo el equipo. |
| Objetivo predominante | Detectar defectos y verificar especificaciones. | Prevenir defectos, validar necesidades y obtener feedback temprano. |
| Organización | Mayor separación entre desarrollo y testing. | Colaboración continua entre desarrolladores, testers y negocio. |
| Automatización | Puede utilizarse, pero no siempre es central. | Es fundamental para sostener pruebas y entregas frecuentes. |

**Importante:** en el enfoque tradicional también se pueden diseñar pruebas tempranamente. Agile no elimina las actividades de testing existentes, sino que cambia su organización y frecuencia.

### Manifiesto de Testing Agile

Propone cinco cambios de perspectiva, priorizando:

- **Pruebas durante el desarrollo** sobre pruebas al final.
- **Prevenir defectos** sobre encontrarlos.
- **Entender lo que se está probando** sobre verificar únicamente la funcionalidad.
- **Construir un mejor sistema** sobre intentar romperlo.
- **Responsabilidad de todo el equipo por la calidad** sobre responsabilidad exclusiva del tester.

La idea no es dejar de buscar defectos, sino **incorporar la calidad durante la construcción del producto**, trabajando colaborativamente.

### Principios de Testing Agile

- **Testing se mueve hacia adelante:** comenzar a probar desde las primeras actividades del proyecto.
- **Testing no es una fase:** es una actividad continua e integrada al desarrollo.
- **Todos hacen testing:** la calidad es responsabilidad compartida.
- **Reducir la latencia del feedback:** detectar y comunicar problemas lo antes posible.
- **Las pruebas representan expectativas:** expresan cómo debería comportarse el software.
- **Mantener el código limpio y corregir defectos rápidamente:** evitar acumular problemas y deuda técnica.
- **Reducir la sobrecarga de documentación:** mantener documentación útil, suficiente y actualizada.
- **Las pruebas son parte del Done:** una funcionalidad no se considera terminada si no cumple los criterios de calidad establecidos.
- **De probar al final a desarrollo conducido por pruebas:** utilizar las pruebas para orientar la construcción del software.

### Prácticas de Testing Agile

- **Pruebas unitarias y de integración automatizadas:** permiten comprobar rápidamente los componentes y sus interacciones.
- **Pruebas de regresión de sistema automatizadas:** verifican que los cambios no hayan afectado funcionalidades existentes.
- **Pruebas exploratorias:** permiten investigar el comportamiento del software mientras se aprende sobre él.
- **TDD (Test-Driven Development):** desarrollo conducido por pruebas, escribiendo pruebas antes del código.
- **ATDD (Acceptance Test-Driven Development):** desarrollo conducido por pruebas de aceptación.
- **Control de versiones de las pruebas junto con el código:** permite mantener las pruebas asociadas a la versión correspondiente del software.

### Pruebas exploratorias y de regresión

- **Exploratorias:** el tester aprende sobre el producto mientras diseña y ejecuta pruebas. Ayudan a descubrir comportamientos y riesgos que no estaban previstos.
- **Regresión:** se vuelven a ejecutar pruebas después de realizar cambios para comprobar que las funcionalidades existentes siguen funcionando.

La automatización hace posible ejecutar regresión frecuentemente sin repetir manualmente todos los pasos.

### Rol y responsabilidades del tester ágil

En Agile, el tester deja de ser solamente quien encuentra defectos al final y pasa a ser un **colaborador activo durante todo el desarrollo**.

Sus responsabilidades principales son:

- Participar desde el comienzo en el análisis de requisitos y criterios de aceptación.
- Colaborar con desarrolladores y representantes del negocio.
- Identificar riesgos, diseñar pruebas y realizar testing exploratorio.
- Contribuir a la automatización y a la mejora continua.
- Comunicar resultados y defectos para obtener feedback temprano.

**Idea fundamental:** el tester aporta conocimientos especializados, pero **no es el único responsable de la calidad del producto**.

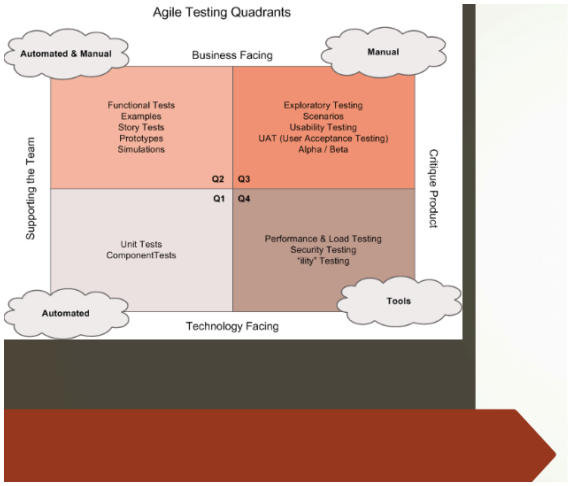
### Cuadrantes de Testing Ágil

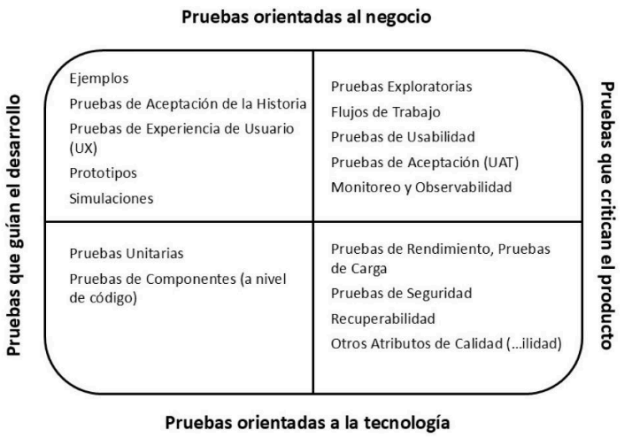
Los cuadrantes permiten **clasificar y seleccionar las pruebas según su propósito y orientación**. Se organizan mediante dos dimensiones:

- **Orientación:** pruebas dirigidas al negocio o a la tecnología.
- **Propósito:** pruebas que apoyan el desarrollo o que evalúan críticamente el producto.

| Cuadrante | Propósito | Ejemplos |
|---|---|---|
| **Q1 — Tecnología / Apoyar desarrollo** | Comprobar la implementación técnica. | Pruebas unitarias y de componentes. |
| **Q2 — Negocio / Apoyar desarrollo** | Expresar y verificar el comportamiento esperado. | Pruebas funcionales, ejemplos y pruebas de aceptación de historias. |
| **Q3 — Negocio / Criticar producto** | Evaluar el producto desde la experiencia del usuario. | Exploratorias, usabilidad y aceptación de usuario. |
| **Q4 — Tecnología / Criticar producto** | Evaluar características técnicas y atributos de calidad. | Rendimiento, carga, seguridad y recuperabilidad. |

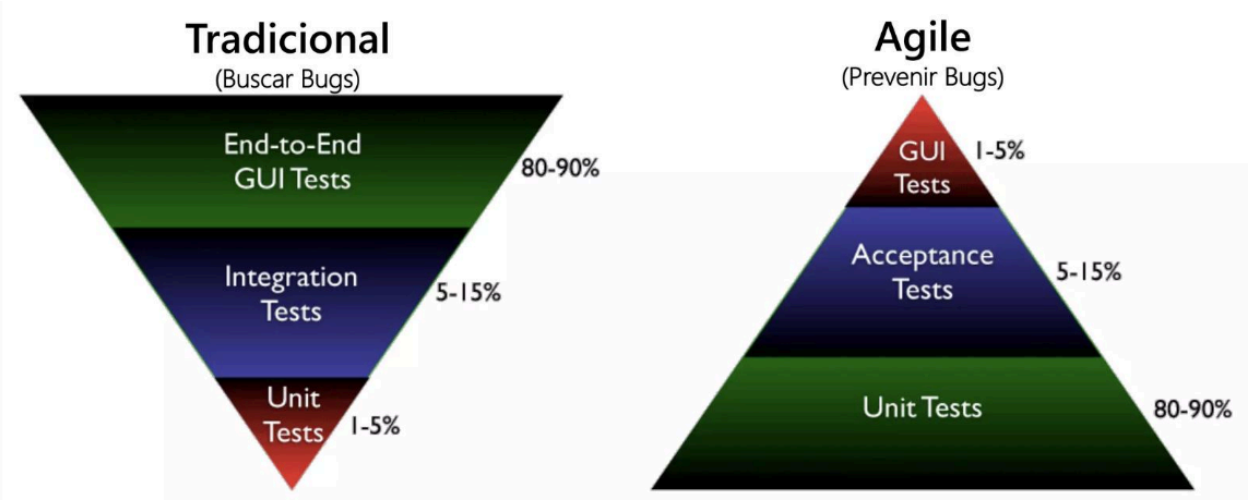
**Importante:** los cuadrantes no son niveles ni fases de testing. Se utilizan para identificar qué clases de pruebas necesita el producto, y no establecen un orden obligatorio de ejecución.




## Pirámide del Testing Agile

La pirámide representa una estrategia para **distribuir las pruebas, priorizando las más rápidas y económicas de ejecutar y mantener**.




Se organiza en tres niveles:

- **Base — Pruebas unitarias y de componentes:** son numerosas, rápidas, focalizadas y generalmente automatizadas. Comprueban pequeñas partes del código.
- **Nivel intermedio — Pruebas de servicios/API y reglas de negocio:** verifican interacciones y comportamientos funcionales sin depender necesariamente de la interfaz gráfica.
- **Cima — Pruebas de interfaz y flujos completos:** comprueban recorridos del usuario a través de la aplicación. Suelen ser más lentas, frágiles y costosas de mantener.

Las pruebas manuales y exploratorias complementan la pirámide.

**Diferencia con el enfoque tradicional:** en la representación utilizada por la profesora, el testing tradicional concentra más esfuerzo en las pruebas de interfaz y de extremo a extremo. Agile busca invertir esa distribución, construyendo una base sólida de pruebas unitarias automatizadas.

### Automatización del testing

Consiste en utilizar herramientas y scripts para **ejecutar pruebas, comparar resultados y detectar problemas automáticamente**, reduciendo la intervención manual. Beneficios:

- **Rápida ejecución:** permite probar frecuentemente.
- **Reusabilidad:** las mismas pruebas pueden ejecutarse sobre nuevas versiones.
- **Mayor cobertura:** facilita comprobar más escenarios de manera repetible.
- **Mayor precisión:** reduce errores humanos durante tareas repetitivas.
- **Mayor alcance:** permite ejecutar pruebas sobre diferentes configuraciones y versiones.
- **Reducción de costos:** disminuye el esfuerzo de ejecución repetitiva a largo plazo.

La automatización favorece la **Integración Continua (CI)**, ya que permite ejecutar pruebas automáticamente cuando se incorporan cambios al código.

**No significa automatizar absolutamente todas las pruebas:** las exploratorias, las evaluaciones de usabilidad y otras actividades que requieren juicio humano siguen siendo necesarias.

### Desafíos culturales del Testing Agile

Adoptar Agile implica modificar la forma en que las personas trabajan y comprenden la calidad. Los principales desafíos son:

- **Cultura organizacional:** pasar de buscar culpables por los defectos a aprender de ellos y prevenirlos.
- **Gestión del cambio:** abandonar la separación rígida entre desarrollo y testing, incorporando colaboración y automatización.
- **Responsabilidad compartida:** comprender que la calidad pertenece a todo el equipo y no solamente al tester.
- **Manejo de expectativas:** acordar con el negocio qué significa que el producto está terminado y qué nivel de calidad se espera.
- **Adaptación de roles:** desarrollar nuevas habilidades de comunicación, colaboración, negocio y automatización.

**Conclusión:** el Testing Agile no consiste simplemente en probar más rápido o automatizar pruebas. Su objetivo es integrar la calidad al desarrollo mediante prevención, colaboración y retroalimentación continua.

# 4. Parte práctica: Testing de Caja Negra

## 4.1. Partición de equivalencias

### 4.1.1. Fundamentos

- ¿Qué es la partición de equivalencias?
- ¿Para qué se utiliza?
- Clases válidas e inválidas.

### 4.1.2. Clases de equivalencia de entrada

- Identificación de condiciones externas.
- Rangos de valores continuos.
- Valores discretos.
- Selección simple.
- Selección múltiple.
- Definición de clases válidas e inválidas.

### 4.1.3. Clases de equivalencia de salida

- Identificación de resultados posibles.
- Salidas válidas.
- Salidas inválidas o respuestas de error.
- Relación entre entradas y resultados esperados.

## 4.2. Diseño de casos de prueba

### 4.2.1. Derivación de casos de prueba

- Seleccionar valores representativos de las clases.
- Combinar clases de equivalencia cuando corresponda.
- Cubrir las clases válidas e inválidas.
- Buscar una cantidad mínima de casos de prueba.
- Asegurar la cobertura de los criterios de aceptación.

### 4.2.2. Estructura de un caso de prueba

- ID del caso de prueba.
- Prioridad.
- Nombre.
- Precondiciones.
- Pasos.
- Resultado esperado.

## 4.3. Procedimiento para resolver el parcial práctico

### 4.3.1. Resolución paso a paso

1. Leer la User Story y sus criterios de aceptación.
2. Identificar las condiciones externas.
3. Definir las clases de equivalencia de entrada.
4. Definir las clases de equivalencia de salida.
5. Seleccionar los casos de prueba necesarios.
6. Completar los casos en la plantilla.
7. Verificar que todos los criterios de aceptación estén cubiertos.